Firmware Implementation for the Proto-DUNE II Hit Finding Architecture with the Xilinx HLS Tool

MPhys project report by Joshua Horswill

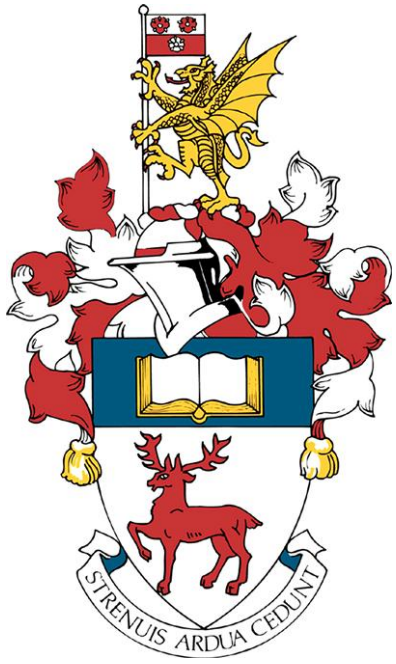
Department of Physics and Astronomy
University of Southampton

Student ID: 29150914

Supervised by K. Manolopoulos

Particle Physics Department
STFC Rutherford Appleton Laboratory

May 18, 2021



Abstract

The Deep Underground Neutrino Experiment (DUNE) is a large-scale international particle physics project based in North America. It will consist of two underground neutrino detectors placed within the path of an intense neutrino beam fired from Fermilab. The far detector will be 1300 kilometres from the source and 1500 metres underground, in the Sanford Underground Research Facility (SURF). The near detector is positioned 574 metres from the beam source. Objectives of the experiment include the study of certain properties of charge-parity violation (CPV) in the lepton sector, as well as the analysis of proton decay, and the investigation into black hole formation through supernova neutrino detection [1]. The focus of this study is the hit finder architecture contained within the 'Trigger Primitive Generation' (TPG) cores of ProtoDUNE II. The project objective was to replace existing firmware algorithms written for the detector's 'Field-Programmable Gate Array' (FPGA) boards with designs synthesised via Xilinx's Vivado 'High Level Synthesis' (HLS) software. C++ code was used to implement the desired behaviour. This enabled a comparison of the performance and utilization reports of the new designs and those written manually in a hardware description language (HDL) called VHDL. HLS has the potential to make firmware design more accessible to a wider range of scientists. This could lead to an increased pace in both the development of FPGA-based detector algorithms and the acquisition of important data.

Contents

List of Figures

1 Introduction

1.1 DUNE Experiment Goals and Context

The DUNE collaboration is an amalgamation of many neutrino-physics communities, centred around an opportunity provided by the large investment of the U.S. Department of Energy. This will contribute to a large expansion of the underground experimental architecture of the SURF facility, as well as a megawatt neutrino-beam facility positioned 1300km across North America at Fermilab. This investigation will be groundbreaking for several areas of physics, including neutrino oscillation studies, searches for proton decay, and for neutrino-based astrophysics i.e. black hole and supernova burst phenomena. Section 1.2 discusses the physics behind the experiment, and how DUNE could tackle key theoretical questions in the neutrino sector.

1.2 The Importance of Neutrinos in the Standard Model

The last particle predicted by the standard model (SM) to be experimentally confirmed was the Higgs boson. It was eventually discovered in 2012 by the ATLAS and CMS groups at the CERN Large Hadron Collider experiment in Geneva. This was a milestone of the predictive success achieved by the SM, and opened up a new saga in the exploration of physics beyond the SM (BSM). Physicists have not yet seen success when searching for BSM particles. In this way, the SM has agreed particularly well with experimental results.

The current formulation of the SM was finalised in the 1970s, after the existence of quarks were experimentally confirmed [2]. Since then, all the remaining undiscovered particles predicted by the model have been confirmed, such as the top quark in 1995 [3], the tau neutrino in 2000 [4] and of course the Higgs boson [5]. An especially accurate prediction of the properties of the weak interaction as well as the W and Z bosons have further validated the SM. Now that all SM particles have been established, it is useful to consider the benefits of investigating BSM sectors of physics.

There are still certain natural phenomena that remain to be unexplained such as the strong CP problem[1], neutrino oscillations (discussed in section 1.2.2), and the occurrence of dark matter and energy. The SM also fails to meet the requirement of being an all-encompassing, coherent theoretical framework of physics. These conditions are not satisfied because important issues such as gravitation, the accelerating expansion of the universe, or the asymmetry in the relative abundance of matter and antimatter are not accounted for. Inquiries into the BSM sector could lead to solutions for these issues.

Analysing the properties of neutrino mixing and mass could provide a strong opportunity to discover evidence for BSM physics. This is because recent experimental data contradicts the current SM construction that represents neutrinos

[1]This is a contradiction between experiment, where CP symmetry is never violated by quantum chromodynamic interactions, and theory, where the current mathematical formulation of strong interactions state that it is possible for CP violation to occur.

as massless particles. The reason for this representation comes from the fact that there was no need for neutrinos to be massive when the SM was built. Fermion masses are not predicted by the SM, and are instead set as input parameters in the Lagrangian. Therefore the SM neutrino mass parameter can be changed to agree with experimental evidence.

It would be wrong to say that the SM does not allow for massive neutrinos. It is possible to introduce an interaction term in the Lagrange density, which uses only the standard-model multiplets and generates Majorana masses[2] for the neutrinos. However, this term is nonrenormalisable and violates Baryon minus Lepton number (B-L) conservation [6]. Adding right-handed neutrinos to the SM would fix this issue, and this idea is discussed further in section 1.2.3.

A wealth of data has been recorded since the dawn of the 21st century, kickstarted by the first observation of the disappearance of atmospheric muon neutrinos in 1998 [7, 8]. As a result, it has since been confirmed that neutrinos oscillate in their flavour. A crucial conclusion can be made from this, which is the fact that neutrinos have mass, although unexpectedly small; the sum of the three neutrino mass eigenstates must be less than one millionth of an electron mass eigenstate [9, 10]. This conclusion is drawn from the fact that a fundamental requirement of neutrino oscillation is that the mass eigenstates of each neutrino are unequal to one another. In other words, the mass eigenstate cannot be the same as the flavour eigenstate. Section 1.2.2 discusses this in more detail.

The origin of these small neutrino masses and their oscillations is still not confirmed however, and is up for discussion. DUNE exists to carry out an investigation of neutrino oscillations to test 'Lepton-sector Charge Parity Violation' (LCPV) as well as the ordering of neutrino masses and the three-neutrino paradigm[3]. Before this is explored however, some preliminary concepts must be discussed.

1.2.1 Majorana versus Dirac Neutrinos

It is still not confirmed whether neutrinos are Majorana or Dirac particles. Majorana particles and their antiparticles differ only in their chirality, whereas Dirac particles are not at all like their antiparticle counterparts. All SM fermions besides the neutrino are known to be of the Dirac variation [12]. In the 1957 Wu experiment an observation of parity violation in cobalt-60 beta decay was made, leading to the conclusion that neutrinos must be 'left-handed' and antineutrinos 'right-handed' [13]. This is called helicity, where right and left-handed spins denote a spin vector that is parallel and antiparallel to the momentum vector, respectively.

This is not to be confused for a chiral phenomenon, where an entity is not identical to its mirror image. Chiral theories are those which are asymmetric with respect to chiralities i.e. does not respect parity symmetry. For massless

[2] Defined in section 1.2.1.

[3] The quantum mechanical mixing of the three neutrino mass eigenstates producing the three known neutrino flavour states [11]

particles chirality is the same as helicity. Massive particles on the other hand can have their helicity reversed by changing the frame of reference so that the momentum vector is moving opposite to the spin vector. It is not Lorentz invariant, and chirality and helicity must be distinguished. Therefore, the Wu experiment is not a confirmation that neutrinos are different to antineutrinos since their helicities can be reversed by changing the observational reference frame.

One might wonder how the existence of Majorana neutrinos could be proven. Firstly, the conservation of total lepton number could be analysed, where the total lepton number for a system is given by $L = n_l - n_{\bar{l}} = L_e + L_\mu + L_\tau$, where n_l is the number of leptons, $n_{\bar{l}}$ is the number of anti-leptons and $L_{flavour}$ denotes the individual lepton number for a given flavour. Lepton flavour is not conserved in neutrino oscillation, but so far the SM supports total lepton number conservation. If there is an experimental observation of the nonconservation of the total lepton number due to the neutrino masses, then this would support the theory that neutrinos are Majorana type [14]. Majorana neutrinos would violate both total lepton number and B-L conservation via neutrinoless double beta decay [15]. This interaction is only possible if neutrinos are of the Majorana variety and have at least one type with non-zero mass, the latter of which has already been established. Neutrinoless double beta decay is a process of great interest, being searched for by a significant number of experiments [16–19]. Regardless, it should be explained how chirality and B-L conservation come into play when discussing baryon asymmetry.

To answer this, it would be useful to review the Sakharov conditions. In 1967, Andrei Sakharov proposed three necessary conditions for baryogenesis to produce matter and antimatter at different rates [20]. He was influenced by evidence of cosmic background radiation and a recent discovery of CP violation in neutral kaon decay (described in section 1.3). The conditions are as follows:

1. Baryon number violation.

   This is a natural result of producing an excess of baryons over anti-baryons

2. C and CP-symmetry violation.

   Charge conjugation violation ensures that interactions producing more baryons than anti-baryons will not be counterbalanced by the opposite case, where more anti-baryons are produced. CP violation makes sure that unequal numbers of left-handed baryons and right-handed anti-baryons are produced. The same goes for unequal production of left-handed anti-baryons and right-handed baryons.

3. Interactions occurring outside of a thermal equilibrium.

   If this was not the case, then CPT symmetry would compensate for processes increasing and decreasing the baryon number. The rate of the reaction causing baryon dominance must be less than the expansion rate of the universe. In this way, the baryons and anti-baryons never achieve thermal equilibrium.

Following this proposal, a substantial number of physicists believe that the matter-antimatter asymmetry is caused by chiral anomalies [21]. Chiral anomalies are theoretical instances where the nonconservation of the total left and right-handed chirality in a system occurs. This is a reason why the lepton number conservation law could be violated. These events are prohibited by B-L conservation, but this law is contradicted by evidence of charge-parity violation. Hence their relevance in the investigation into baryogenesis.

Neutrino oscillation is a mechanism within the electroweak model that makes it a chiral theory, and is discussed in section 1.2.2.

1.2.2 Neutrino Oscillation

Neutrino oscillation is the quantum mechanical transformation of a neutrino created with a specific lepton family number into one with a different family lepton number. This phenomena be measured when the neutrino propagates through space. The probability of oscillation is proportional to the ratio of D/E, where D is the distance travelled and E is the energy of the neutrino [22]. The phenomena was established experimentally by the Super-Kamiokande and Sudbury neutrino observatories. This discovery was recognised by the 2015 Nobel prize for physics.

The oscillation phenomena appears due to the mixing of neutrino flavour and mass eigenstates. Previously, the neutrino state produced by weak decay was portrayed as a linear superposition of mass eigenstates with equal energy or equal momenta. This was corrected by describing the neutrino as entangled with the other particles emerging from the decay. For practical purposes, one can describe a travelling neutrino state as a mass eigenstate [23]. As the superposition of neutrino mass eigenstates propagates through space, the quantum mechanical phases of each eigenstate change at slightly different rates due to their mass differences. A neutrino born as one species will over time become a superposed mixture of e, μ and τ neutrino states.

This is the property that makes flavour oscillation impossible without neutrino mass eigenstates that possess unequal mass. Hence, evidence of flavour oscillation confirms massive neutrinos.

The mass differences between neutrino flavours are small compared to their long coherence lengths[4]. Macroscopic travel distances are required to observe this oscillation effect [24], such as the long baseline of the DUNE experiment. Section 1.2.3 discusses one of the objectives of DUNE: searching for the mechanism responsible for the mass of neutrinos.

1.2.3 Neutrino Mass Origins and Sterile Neutrinos

It is important to note that many investigations into neutrino physics involve BSM concepts which are contradictory to the SM. Consider the fact that right-handed neutrino singlets are not predicted by the SM [14]. This indirectly

[4]The coherence length is the distance at which a quantum mechanical wavefunction maintains a specified degree of coherence.

suggests that neutrinos are massless, yet as mentioned before, experimental observation of neutrino oscillation contradicts this.

However, this does not mean the SM cannot be modified to accommodate massive neutrinos. One can add a right-handed Lagrangian term to the SM in order to remedy this. If neutrinos have Dirac masses, then an SU(2) singlet (sterile neutrino) could be included, and the mass would be generated via the Dirac mechanism. If neutrinos have Majorana masses, they can be explained via the Majorana mechanism [25].

However, it is not known for certain how neutrino masses arise. In the SM, fermions have mass due to interactions with the Higgs field, which involves left and right-handed versions of each fermion. This means that right-handed neutrinos are theoretically well motivated, since all fermions have been observed with left and right chirality [26].

The huge disparity in mass scales between neutrinos and the rest of fermions also remains to be explained. A possible solution to this issue could be the existence of a sterile neutrino, yet only left-handed neutrinos have been observed so far. This explanation involves a Majorana mass term for right-handed neutrinos that interact with the left-handed versions. The interaction would be analogous to the Higgs field interactions with the rest of the SM fermions. Right-handed neutrinos are predicted to be heavy since they would induce a very small mass for the 'active' left-handed variety. Active neutrinos would be proportional to the reciprocal of the heavy mass. The right-handed neutrinos would have to possess an energy close to the GUT scale (of order 10^{16} GeV) to reflect the neutrino mass limits that have been cosmologically determined. This is called the 'Seesaw mechanism' [27].

The term 'sterile neutrino' is used to differentiate right-handed neutrinos or left-handed antineutrinos from the active kind. Active neutrinos possess an isospin charge of $\pm\frac{1}{2}$ and interact weakly as a result. The sterile kind only interact via Yukawa interactions with ordinary leptons and Higgs bosons, which means they influence and are influenced by the gravitational field and can mix with ordinary neutrinos through the Higgs mechanism[5]. The sterile neutrino is Majorana type because it would have the same weak hypercharge, weak isospin and electric charge as its antiparticle, since all of these values are zero and therefore unaffected by sign reversal. These combined factors make right-handed neutrinos promising candidates for dark matter [26]. Linking back to section 1.2.1 one can associate sterile neutrinos with two contradicting outcomes. The first is that neutrinos and antineutrinos differ only in chirality. The second is that sterile neutrinos exist as separate particles from the common left-handed neutrinos and right-handed antineutrinos.

Now that some context has been provided, these concepts will be tied into one of the most important searches in BSM physics.

[5]The spontaneous breaking of electroweak symmetry during interactions, causing the bosons interacting with the Higgs field to have mass, when they otherwise would not.

1.3 Charge Parity Violation and Baryon Asymmetry

Cosmology tells us that at the beginning of the universe, there existed equal amounts of matter and antimatter. This contrasts the current asymmetrical composition, suggesting there must be differences in the behaviour between baryonic and antibaryonic matter. As outlined by the second Sakharov condition, the symmetry of charge conjugation $\times$ parity must be violated. This means that in a system containing CP-violating matter, if all particles were replaced by their antiparticles and their coordinates were 'inverted' by a parity operator, then the behaviour of the system would change. One of the first instances of observed CP violation is the decay of a neutral kaon to two pions: $K_S^0 \rightarrow \pi^+ + \pi^-$ [28]. Kaons have been observed mixing between the particle and antiparticle versions of themselves via CP-invariant oscillations. The decay of a Kaon into pions with $\mathrm{C} \times \mathrm{P} = \pm 1$ means that the initial and final states were different. This was later understood in terms of the dynamics of quarks.

Since this observation, CP violation has been recorded in the strange, beauty and charm quark sectors. All conclusive data is consistent with the theory that CP violation is caused by a single complex phase in the unitary quark-mixing matrix. This is also known as the Cabibbo-Kobayashi-Mashawa (CKM) matrix. The CKM matrix gives the amplitude for a negatively-charged quark to convert through a weak interaction into a positively charged quark. Another way of looking at it is that each CKM element contains information on the strength of flavour-changing weak interactions involving freely-propagating quarks [29]. See figure 1 for the generalised version of the CKM conversion equation.

Four independent parameters are required to fully define the CKM matrix. One of the most common parameterizations is the Wolfenstein parameterization. Within this framework there exists two parameters ρ and η, whose magnitudes determine the complex phase of the matrix [30]. They account for quark behaviour, including CP violating phenomena, with great success.

However, despite observing CP violation in QCD interactions, it would be incorrect to assume that baryon asymmetry has been accounted for. In actual fact, the current theoretical frameworks that link early-universe CP violation to present day quark CP violation yields an asymmetry that is too small compared to what is actually seen in the universe [32]. This would lead to the thinking that CP violation in the lepton sector may take responsibility for the dominance of baryonic matter, through leptogenesis.

Leptogenesis is a scenario in which heavy neutral leptons, which are their own antiparticle, existed briefly before undergoing a CP-violating decay. It is a possible explanation for the observed composition of matter [33]. These heavy neutral leptons are predicted by the seesaw models mentioned in section 1.2.3.

To record the rate of CP violation in the lepton sector, one must search for the difference in behaviour between neutrinos and antineutrinos. The current approach is to compare the rates for two CP mirror-image processes, an example of which is illustrated in figure 2. Not only is each particle converted to their antiparticle, but their chirality is also reversed. Regardless of whether neutrinos are Majorana or Dirac, if the rates are unequal between processes A and B, there

$$\begin{bmatrix} d' \\ s' \\ b' \end{bmatrix} = \begin{bmatrix} V_{ud} & V_{us} & V_{ub} \\ V_{cd} & V_{cs} & V_{cb} \\ V_{td} & V_{ts} & V_{tb} \end{bmatrix} \cdot \begin{bmatrix} d \\ s \\ b \end{bmatrix}$$

Figure 1: The CKM matrix equation
This equation demonstrates the conversion of the mass eigenstates of the original negatively charged quarks into the weak interaction doublet partners of said negatively charged quarks via the CKM matrix. d, s and b denote the down, strange and bottom quarks respectively. These are referred to as 'down-type' quarks. Similarly, u, c and t denote the up, charm and top quarks respectively. They are referred to as 'up-type' quarks. Each coefficient squared $|V_{ij}|^2$ is proportional to the probability that the jth down-type quark decays into the ith up-type quark. The q' down type quarks on the left hand side denote the weak interaction doublet partners and the q down type quarks on the right hand side denote the original mass eigenstates [31].

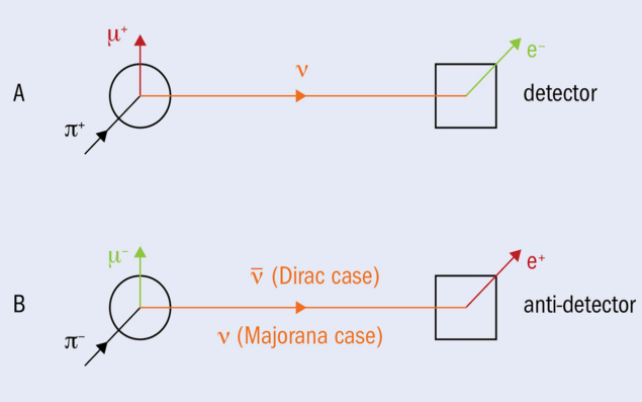


Figure 2: This figure shows the comparison of two CP mirror-image processes. In process A, a π^+ decays into a μ^+ and a muon neutrino, that travels a macroscopic distance from the source to a detector, generating an electron from an interaction [34]. In process B, the same interaction occurs but each particle is replaced by it's antiparticle and the chirality is reversed. The 'anti-detector' in reality is impossible to implement, but the same detector can be used in both processes if the difference between the cross sections in process A and B are taken into account. The probability for the muon neutrino to produce an electron at the detector oscillates as a function of distance travelled over the particle's energy.

$$\begin{bmatrix} \nu_e \\ \nu_\mu \\ \nu_\tau \end{bmatrix} = \begin{bmatrix} U_{e1} & U_{e2} & U_{e3} \\ U_{\mu 1} & U_{\mu 2} & U_{\mu 3} \\ U_{\tau 1} & U_{\tau 2} & U_{\tau 3} \end{bmatrix} . \begin{bmatrix} \nu_1 \\ \nu_2 \\ \nu_3 \end{bmatrix}$$

Figure 3: PMNS matrix equation
This equation demonstrates the leptonic equivalent of the CKM matrix equation. The first term represents the neutrino flavour eigenstates. The second term is the PMNS matrix. The third and final term represents the neutrino mass eigenstates. The elements U_{l1}, U_{l2}, U_{l3} act as the coefficients for the eigenstate of flavour l to be given in terms of a linear combination of the mass eigenstates $1, 2$ and 3 [36].

is a measurable CP invariance of the weak interactions of leptons.

Similarly to the fact that quark CP violation arises from a complex phase in the quark-mixing CKM matrix, leptonic CP violation in neutrino oscillation can arise from the complex phase in the leptonic analogue. This is represented by the unitary Pontecorvo-Maki-Nakagawa-Sakata (PMNS) matrix. The PMNS matrix contains information on the mismatch of quantum states of neutrinos when they freely propagate over a distance and take part in weak interactions [35]. The PMNS matrix is part of a neutrino oscillation model. One could question the relevance of the PMNS matrix within oscillation physics and the DUNE experiment. To answer this, we must consider the parameterization of the PMNS matrix.

Neutrinos only interact via the SM charged-current and neutral-current weak interactions. The mass eigenstates can be defined as ν_1, ν_2, ν_3 with respective masses m_1, m_2, m_3. As mentioned before, they are distinct from the charged-current interaction eigenstates (flavour eigenstates) which can be denoted as ν_e, ν_μ, ν_τ. These are labelled according to the charged-lepton that they couple with during a charged-current weak interaction[6]. The mass eigenstates can be expressed as linear combinations of the flavour eigenstates and vice versa [36]. This is shown in the eigenstate mixing equation in figure 3, where the coefficients of the linear combinations make up the elements of the PMNS matrix.

If the neutrinos are Dirac type, the PMNS matrix can be parameterized, like the CKM matrix, by three mixing angles:

$$\sin^2(\theta_{12}) = \frac{|U_{e2}|^2}{1 - |U_{e3}|^2}$$

$$\sin^2(\theta_{23}) = \frac{|U_{\mu 3}|^2}{1 - |U_{e3}|^2}$$

$$\sin^2(\theta_{13}) = |U_{e3}|^2$$

and one complex phase:

[6]Mediated by the W± bosons.

$$\delta_{CP} = \text{-arg}(U_{e3})$$

These parameters are being measured to greater precision in the DUNE experiment in order to better understand neutrino oscillation. δ_{CP} especially is not well known. Figure 4 visualises the probability of neutrino oscillation at the baseline of the DUNE experiment for different values of the complex phase. One can see that measuring the cross section statistics for this process at certain neutrino energies could give insight into the value of δ_{CP}.

If the neutrinos are Majorana type there would be two other physical complex phases. However, for practical purposes neutrino oscillation experiments cannot distinguish Majorana from Dirac neutrinos.

Current collaborations measuring neutrino oscillation and searching for leptonic CP violation include the NOvA and T2K experiments. The DUNE and Hyper-Kamiokande projects will continue to probe this concept with more sensitivity and precision.

The next segment will explore how DUNE in particular will practically approach finding out if CP symmetry is unique to just quarks, or to all constituents of matter.

1.4 An Overview of the DUNE Experiment

In order for DUNE to achieve its goals, three central components are required. The first is a high-intensity neutrino source generated from a megawatt-class proton accelerator at Fermilab. The second is a large-scale underground far detector situated across the country at the SURF facility. Lastly, DUNE requires a composite near detector installed 574 metres from the neutrino source. To achieve the required precision on its measurements, the DUNE experiment will need to collect the data from several thousand neutrino interactions over the period of ten years. The number of interactions is influenced by the following:

- The intensity of the neutrino beam.
- The probability that the neutrino will oscillate within the baseline (about 2%).
- The cross section of the interaction within the detector.
- The mass of the detector.

The highest power proton beam that a target can sustainably withstand is 1-2 MW, which practically limits the maximum neutrino intensity. Therefore the detector mass must be in the tens-of-kilotons range to achieve the required number of interactions. The cryostats stored within the far detector will hold approximately 17.5kt of liquid argon, reaching a total mass of 70kt per cryostat [38].

Not only must a huge mass of cryostats be used at the far detector, but they shall also be placed 1500m underground at SURF. This is because the rate of

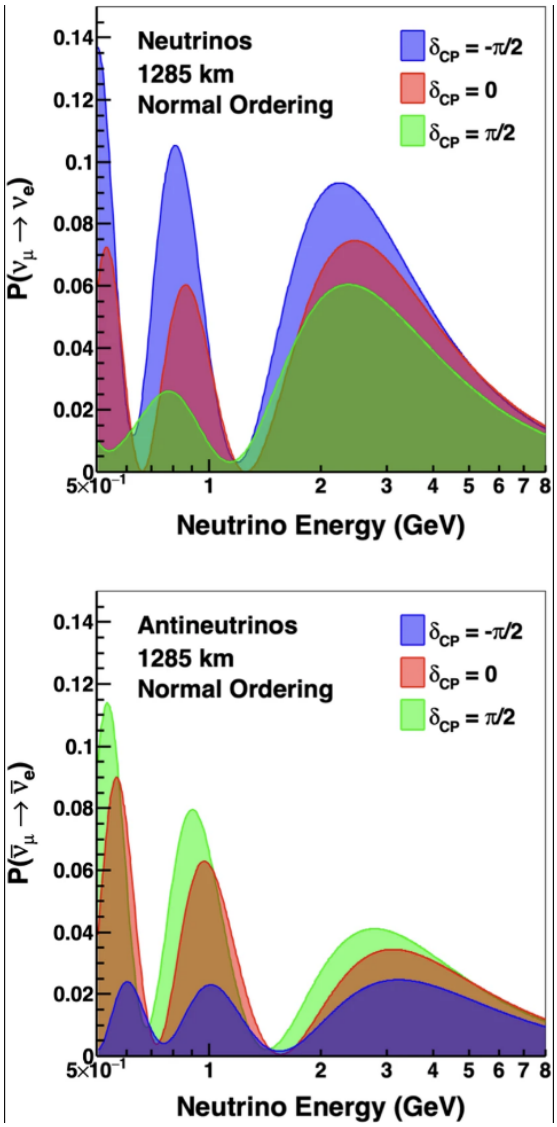


Figure 4: The probability that muon neutrinos (top) and antineutrinos (bottom) will appear at the far detector as electron neutrinos or antineutrinos respectively, depending on the complex phase value from the PMNS parameterization. The y-axis displays the appearance probability and the x-axis displays the neutrino energy [37].

cosmic rays at the Earth's surface is 165 kHz, which would mean that 1 in 10^6 events would be due to neutrinos. This would make the achievement of physics goals related to neutrino oscillation unfeasible. At the planned depth of the far detector, the ratio of neutrino events to cosmic ray events would become slightly higher than 1, making the objective far more obtainable. If these requirements are met, one would wonder how DUNE will achieve a required uncertainty of only a few percent. This seems to be obstructed by the fact that the far detector would only be able to measure the neutrino fluxes to an uncertainty of 10% and the interaction rates to around 20%.

The near detector reduces these uncertainties by measuring neutrinos near the beam source in addition to the far detector data, so that physics measurements can be extracted from differences between the two. Another way to optimise measurements is to place the far detector in the optimal distance range. This range would optimise the determination of the neutrino mass hierarchy and the complex phase parameter δ_{CP}. The range is between 1000 and 2000 km, as shorter baselines have lower optimal neutrino energies so that observables are too low-energy to be visible. Charge parity sensitivity is also reduced by the ambiguity from unknown mass ordering. At longer baselines, CP sensitivity is damaged by matter effects that increase proportionally to distance travelled by the neutrino [38].

Section 1.5 will explore the data acquisition system (DAQ) of the Proto-DUNE detectors.

1.5 ProtoDUNE DAQ System

DAQ architecture can be split into two parts: the front-end and the back-end. The front-end is comprised of circuitry that acquires the data stream from readout-electronics inside the detector, so that it can process and deliver the stream to the back-end: an offline storing, event-reconstruction computation farm. The focus of this report will be on the front-end, including the detection modules and the interface to the detector electronics.

There are several main factors that influence the design of the DUNE DAQ system. One of which is the hardware and maintenance cost per input channel, which is related to the technology life-cycle of each component. This contributes to the need for long-lasting essential parts such as the Xilinx[7] FPGAs and network components. DUNE is set to run for approximately 20 years; the reliability and maintenance of the detector components is more important when compared to other large-scale projects. The power consumption of the detector modules also requires consideration due to cooling contingencies and the variance in running costs. Modularity of the hardware and firmware blocks is emphasised, so that one change in a sub-module would not affect the rest. This emphasis on customisability carries over to the HLS design process. If the design tools are more abstract and are rooted in commonly used languages such as C++, this

[7]A technology company based in the United States that is a primary supplier of programmable logic devices and associated softwares.

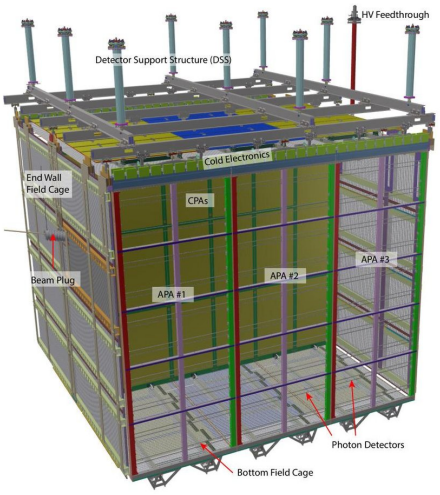


Figure 5: The ProtoDUNE single-phase (SP) liquid argon time project chamber (LAr TPC) module. This contains two drift volumes, separated by a central cathode plane that is flanked by two anode planes, connected to a field cage that surrounds the entire module. Each anode plane is made of three Anode Plane Assemblies (APAs). These connect and send data to the front-end electronics being discussed in this report [40].

will appeal to the vast majority of users (physicists) [39]. With these objectives in mind, R&D and fast firmware modifications can be a quick and efficient process. Section 1.5.1 will explore several types of detectors used in neutrino beam experiments.

1.5.1 Single and Dual Phase LAr TPCs

The use of LAr TPCs is a matured and popular technique used by several short-baseline[8] neutrino experiments. They are popular for several reasons:

- Argon is a noble element of vanishing electronegativity, which means that electrons produced by ionizing radiation will not be absorbed as they drift towards the detector.
- Liquid Argon is inexpensive yet very dense, increasing the likelihood of a particle interacting by 1000 times when compared to Nygren's TPC design [41]. This is important because the cross section for neutrino-nucleon interactions is very small.
- A neutrino-nucleon interaction causes the Argon to scintillate; a secondary energetic charged particle is generated and travels through the liquid,

[8]The baseline refers to the scale of the beam travel distance. DUNE is 'long baseline' as the detector is positioned across the continent from the beam source.

which causes the release of scintillation photons, the number of which is proportional to the energy deposited by the passing particle [42]. This contributes to the reconstruction of the event.

- LAr TPCs offer unprecedented 3D imaging capabilities and the functionality of a homogeneous calorimeter.

There exists both single and dual-phase versions of the LAr TPC modules. Inside SP liquid argon detectors, such as the structure featured in figure 5, the interactions are readout by wires immersed in the liquid and do not require the amplification of the ionized electron charge. The charge is localised using alternative induction and collection wire planes with different orientations. The charges drift horizontally via an induced electric field towards the vertically-placed anode. The SP design has the disadvantage that its electronics must be especially low-noise due to the fact that no signal amplification takes place within the cryostat. However, the SP is a more mature design and has demonstrated excellent performance. These conditions were tested by ProtoDune-SP at CERN [40].

In 2018 ProtoDUNE-SP collected hadron beam and cosmic ray data. When an interaction occurred in the LAr, the value for the total charge deposited along the track is converted to the energy lost from stopping the cosmic ray muons. Additionally, energy deposits were measured from artificially generated beam particles such as muons, pions, protons and positrons. This provided a unique set of high-quality data for the purpose of calibration and experimentalist studies. Hadron-argon cross section measurements are enabled by this new dataset, which is important for future analysis of neutrino oscillation phenomena within DUNE [38].

The dual phase (DP) detectors rely on the amplification of ionisation charges in the ultra-pure cold-argon vapour layer above the liquid. In DP LAr TPCs, ionisation charges produced by neutrino interactions drift vertically towards the liquid-vapour boundary where they are extracted into the gas phase. The charges are then amplified by components named Large Electron Multipliers (LEMs), and collected by segmented anodes. Five photo-multiplier tubes are mounted underneath the TPC drift cage. They detect the Argon scintillation photons (S1) formed from interactions with passing charged particles. They also detect secondary scintillation light (S2) from the electroluminescence of the electrons extracted in the argon vapour. S1 is at least several orders of magnitude higher in detection amplitude than S2 in a tested TPC [43]. In this design, since the signal gain occurs in the gas phase, low-noise electronics are unnecessary.

For both designs, a field cage surrounds and defines the active LAr volumes, ensuring a uniform electric field throughout the cryostat. The DUNE collaboration plans on employing both SP and DP modules in the far detector, but the actual implementation will depend on the combined results from prototype detectors such as ProtoDUNE-SP and ProtoDUNE II.

The next section will describe the interface system which handles the data acquired by the LAr TPCs.

1.6 FELIX

Each LAr TPC module used in the ProtoDUNE II design contains 150 APAs. Every APA accommodates 2560 wires. There are 10 fibres, or data streams, in the APA, filled with 256 wires per fibre. The Front End LInk eXchange (FELIX) system is used to receive the data from the APAs, and was first developed within the ATLAS collaboration. Within this system contains the focus of this project: the TPG core, discussed in section 1.6.2. FELIX is a PC-based networking gateway with an FPGA mounted on PCIe[9] Gen3 cards [44]. This FPGA carries the FELIX firmware that is responsible for receiving, processing and transmitting detector data. Before transmission, this manipulation of data is performed by the hit-finder (HF) architecture. In ProtoDUNE II, FELIX interfaces with and manipulates the data from the APA, during which time it is fed through the TPG core within the HF processor.

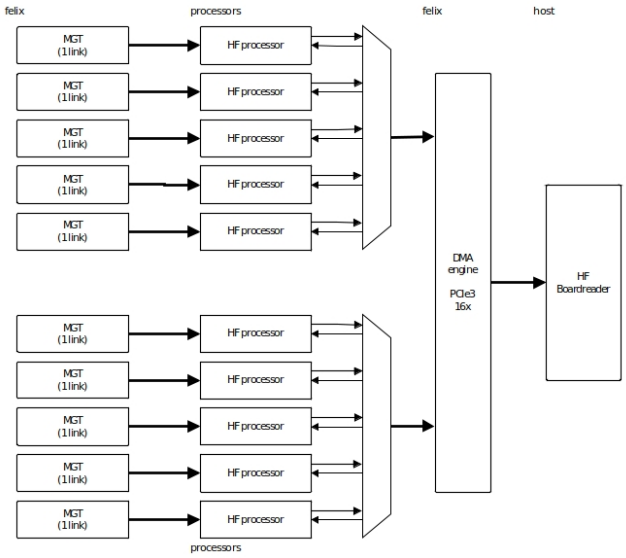


Figure 6: Overview of the FELIX architecture in ProtoDUNE II [45]. 10 MGT links turn optical data into computational bits, which then are fed into the same number of HF processors to detect and record hit packets. The resulting outputs are multiplexed into the CR, processed and finally sent to the back-end DAQ system.

The readout window per neutrino event is between $3 - 5.5$ ms for a 2.25 ms drift time. The FELIX architecture that handles this influx of events consists of 10 components called MGTs (Xilinx's multi-gigabit transceiver modules). The function of an MGT is to process the APA data stream into computational

[9]High speed connection ports used in motherboards to connect GPUs, SSD's, wifi chips and more.

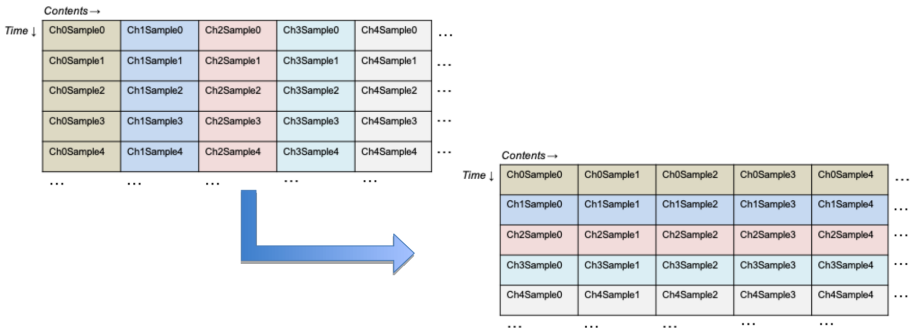


Figure 7: A visualisation of the packet processing operation performed by the data router, before sending reformatted packets to the TPG core [45]. The top left grid shows the arrangement of samples where packets are categorised by the clock tick in which they arrived. The bottom right grid shows the packet arrangement after being processed by the data router, where packets are categorised by the channel in which they arrived, for multiple clock ticks. This is the input described in section 3.1.

bits, which are then fed into 10 HF processors, as seen in figure 6. The hits are produced by processing the incoming ADC (Analog-to-Digital Converted) samples in parallel. The MGT output enters the HFs via the data router, which receives multiple-channel, single clock-tick packets. The packets of ADC values initially exist in a format containing a given sample wave for all channels, until they meet the data router. When packets are processed by the data router they are converted into a format subsisting of all the samples for a given channel i.e. multiple clock ticks of data, as seen in figure 7.

1.6.1 The Data Router

The data router consists of three major blocks: the data reception, data storage and unpacker. The data reception block receives inputs from the MGT and converts the incoming packets from a 32-bit to a 4×16-bit format. This data is then written into a circular buffer (64-bit dual-port RAM) in the data storage block. Data reception includes behaviour that recognises and exploits repeating patterns from incoming data, allowing smarter buffering of smaller parts of the frame. In other words, data reception avoids storing the same information more than once, increasing its efficiency. Channel data from the storage block is then unpacked by the unpacker. Following this, the data is repacked into the desired four-channel format, to be processed by the same number of TPG units. Finally, this data is transmitted via an AXI4[10] bus. The bus data is sent to 10 sets of 4 TPG block processors; one set for each MGT output.

Section 1.6.2 will explain the TPG unit in further detail.

[10]This protocol is defined in section 1.6.2.

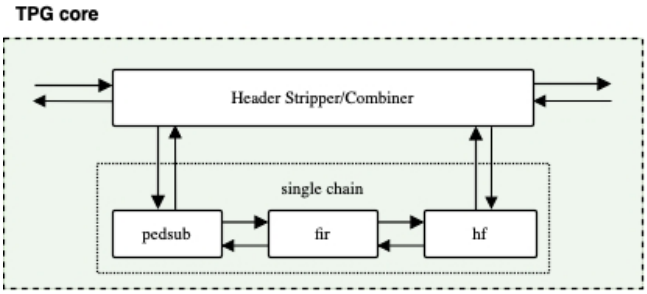


Figure 8: Focus on the TPG core and the sub-block chain, the purpose of which is to filter noise and identify hits from formatted packets received via the data router. [45].

1.6.2 The TPG Core

Each TPG unit processes the data from 64 APA wires, and so there are 4 units for each core[11]. The TPG data consists of a header and a payload. The header determines the source wire, the time stamp, and the origin of the data. The payload is 64 instances of ADC sample data for hit finding. There is an AXI4-stream protocol[12] for the unpacking and reordering of data in the TPG block, featuring 6 general signals for various instructions in each component:

1. `tdata` denotes the 12-bit ADC sample to be processed.
2. `tvalid` denotes if the bus data is valid for processing.
3. `tkeep` goes high when the incoming data should be processed.
4. `tuser` is a flag that denotes the end of a packet frame.
5. `tlast` indicates the end of a packet.
6. `tready` is used to apply back pressure if the processing units ahead cannot process the data fast enough (all throughput is frozen).

Within the TPG core is a mechanism that strips the packet header, called the Header Stripper/Combiner (HSC), featured in figure 8. The header is sent to the output of the block. Meanwhile, the remaining 64 ADC samples and their associated AXI4-S signals are processed by the PedSub (pedestal subtraction), FIR (finite impulse response), and HF sub-blocks in the processing chain. The data router is then signalled to send the next packet forward for input.

[11] $4 \times 64 = 256$, the number of wires in a fiber; ten fibers summed over ten MGTs accounts for all wires within the APA (2560).

[12] A standard interface to connect components that mutually wish to exchange data. The protocol supports multiple data streams that enable upsizing, downsizing and routing operations. The AXI4 protocol is part of an AXI family of micro controller buses, but this one in particular specialises in high performance, memory mapped component requirements.

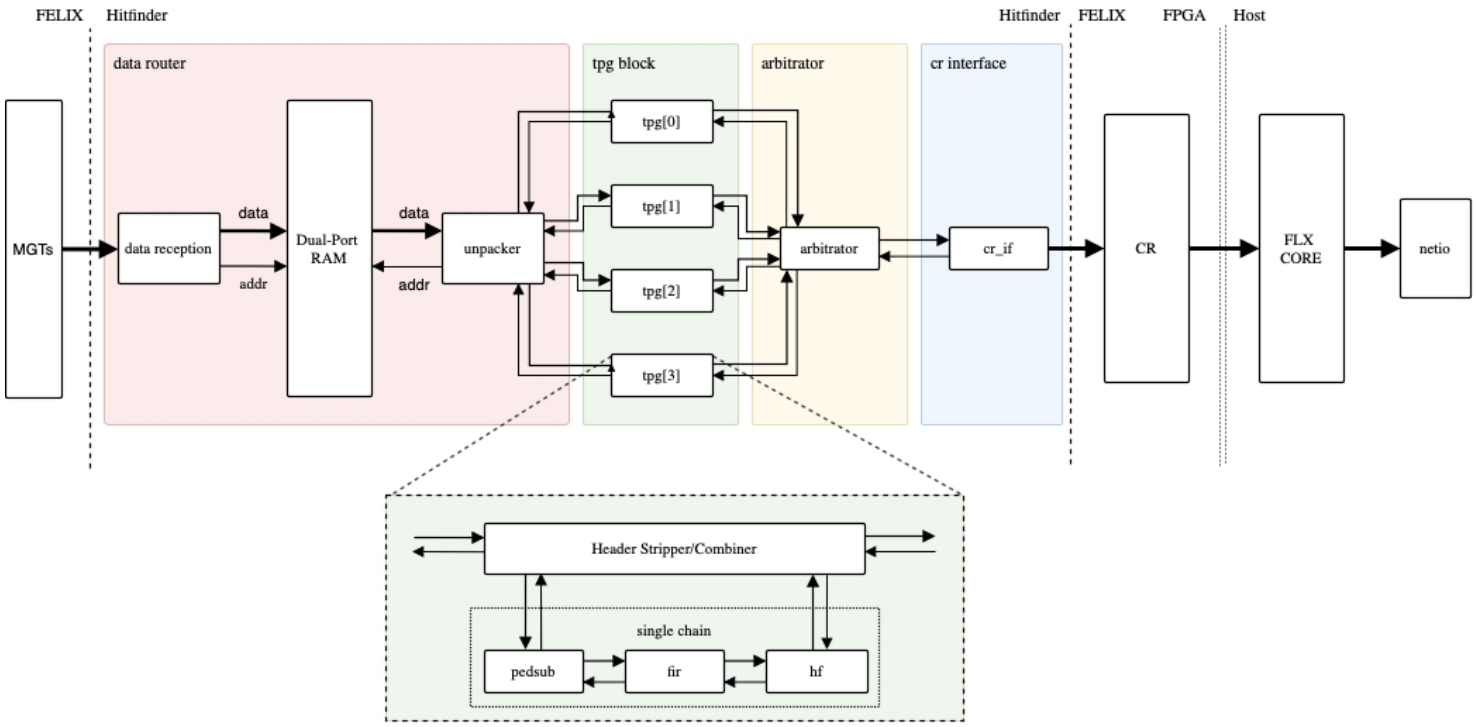


Figure 9: Hit Finder Firmware Architecture [45]. The data router (left) formats packets for processing in the TPG units (centre), the results of which is multiplexed by the arbitrator into the CR interface (right) for additional formatting. Forward arrows represent ADC values and their associated handshake signals, whilst backwards arrows represent back pressure traffic.

The objective of this project is to redesign the firmware algorithms for the TPG sub-blocks in Xilinx's Vivado HLS software suite. This is to acquire a comparison of the performance and resource usage of the HLS design with the existing firmware written by VHDL[13] engineers.

Section 1.6.3 will explain what happens to the output of the TPG core.

1.6.3 CR Interface

The TPG outputs are multiplexed[14] by the arbitrator into the Central Router (CR) interface [39, 46] as seen in figure 9. The arbitrator uses FIFOs (first-in first-out memory) to buffer outputs in the case of multiple incoming packets. This allows the multiplexing of 4 TPG blocks into a single output.

When the data reaches the CR interface, it is again processed by three

[13] VHSIC (very high speed integrated circuit) Hardware Description Language.
[14] Combined into a singular signal output.

main blocks. The first is a FIFO wrapper that prevents back pressure events. These are instances where the rate of data input is faster than the block chain processing rate, so operation must pause to allow time for the processing to catch up. The second block is a hit packet filter which filters out packets which are corrupted or contain zero hits. The final block is the CRIF packer, which translates each hit packet into a format processable by the CR.

Now that a sufficient introduction has been provided for the DUNE experiment and the surrounding physics, the design software will be explored in section 2.

2 HLS

2.1 Background

High-level synthesis (HLS) is an automated design process that interprets an algorithmic description of a desired behavior and creates digital hardware that implements that behavior. It was initially designed to increase the levels of abstraction for design methodologies, forced by the growing capabilities of silicon technology and the increasing complexity of their applications. Beginning in the 1950s, assembly languages were introduced and gradually evolved to high level languages with improved compilation techniques. These languages obeyed the intuitive rules of grammar, syntax and semantics. Gate-level simulations appeared in the 1970s, and in the 1980s progress led to HDLs.

A HDL is a programming language that describes the structure and behaviour of electronic circuits, and more commonly, digital logic circuits [47]. HDLs look like programming languages such as C, but the main difference is that they include the notion of clock timing. The most commonly-used languages are Verilog and VHDL, the latter of which is used in the DUNE firmware framework. HLS allows for a connection between software languages and hardware implementation.

Synthesis begins with a high-level specification of the problem, where behavior is generally decoupled from low-level circuit mechanics such as clock-level timing. Early HLS explored a variety of input specification languages, although recent research and commercial applications generally accept synthesizable subsets of ANSI C, C++, SystemC and MATLAB. C++ was used to design the algorithms featured in this report. The input source code is analysed, architecturally constrained, and scheduled to transcompile into a register-transfer level (RTL) design in a chosen HDL. Following this, the RTL design is commonly synthesized to the gate level by the use of a logic synthesis tool [48].

RTL is a design abstraction which models a synchronous digital circuit in terms of the signal flow between registers and the logic operations being performed on said signals [49]. Hardware registers are circuits composed of flip flops (FFs): a sub-circuit that has two stable states used to store state information. They are a basic storage element in sequential logic, possessing the ability to read or write multiple bits at a time and use an address to select a particular

register. They are used as an interface between the software and the computer peripherals [50].

RTL allows for circuitry simulation and digital analysis, and is used in HDLs to create high-level representations of a circuit. From this, lower-level representations and ultimately actual wiring can be derived. RTL clock cycle data is given in the Vivado HLS performance estimate report. It can determine how long a certain block, loop or function in the code took to complete. This is called the latency. Latency is a crucially important concept in RTL design, as it can make the difference between synthesizing a malfunctioning block or generating a synchronised circuit. Before the firmware block in question can be discussed, the design flow within Xilinx's Vivado HLS software will be explained. This will provide context for the generation of the algorithm.

2.2 Vivado HLS Design Flow

The design flow when approaching the synthesis of a C algorithm is as follows:

1. Compile, simulate, and debug the C algorithm.

   This involves confirming that the C program correctly implements the required processes in the C environment. In order for this to occur, there must be a test bench written in C that includes the top-level function called within a simulation script. This test bench is tasked with recreating the conditions of the implemented firmware and is hosted within `main()`. This typically takes more time to design than the function to be synthesized.

2. Synthesize the C algorithm into an RTL implementation, optionally employing user optimization directives.

   This allocates hardware resources such as Block Random Access Memory (BRAMs), data buses, look up tables (LUTs) [15], digital signal processors (DSPs)[16] and registers to the design. Each operation is scheduled to a certain clock cycle and bound to a functional unit within the board. Variables are bound to storage elements, and the RTL architecture is eventually generated [49]. See section 2.2.1 for more information on directives.

3. Generate comprehensive reports and analyse the design.

   After C synthesis, output files are stored in a directory which contains folders for each RTL output format, and a report folder. The report folder contains files with statistics on the performance and design corresponding to the top-level function and each individual subfunction. The analysis tab allows for cycle-by-cycle insight into the timing of logic operations and the FPGA resource consumption that they require. This is

[15] LUTs are arrays that replace runtime computation with a simpler array indexing operation, saving processing time [51]. FPGAs employ reconfigurable, hardware-implemented, lookup tables to provide programmable hardware functionality.

[16] DSPs are specialised microprocessing chips which are usually employed to measure or manipulate analogue signals. In the case of this project, they are used to perform rapid addition and multiplication calculations [52].

the best opportunity to analyse the effects of applied function directives and `pragma`[17] commands.

4. Verify the RTL implementation using a pushbutton flow.

   This is a post-synthesis validation process that compares the synthesised RTL and C programs to see if they generate the same output; this is called C/RTL co-simulation.

5. Package the RTL implementation into a selection of 'Intellectual Property' (IP) formats.

   This generates the verified VHDL script that can be synthesized via Vivado project manager into an implementable design. Before this can occur, a VHDL wrapper is written to connect the IP block's input and output (IO) ports to the previous and following blocks, ensuring signals are being properly read and written. This is especially important for the AXI4 stream protocol discussed in section 1.6.2. This is because sufficient handshake and boolean signals must enter and leave the block for the AXI4 interface to be able to validate and manage the incoming and outgoing data samples [48].

   Often the block is tested in Questasim[18] to provide a waveform analysis of every port and internal register involved. The Questasim phase is an important verification step in the design process, as the performance of an algorithm within the HLS environment can be vastly different to a VHDL simulation.

When writing HLS source code, there is punishment for using modern modules and storage entities. A simplified, classical style is most easily synthesized. An example of this is that storage arrays must have pre-allocated memory, with a determined number of elements at compile time (ruling out C++ vectors). Additionally, array manipulation modules such as `std::copy` and `memcpy` have very limited functionality or none at all, depending on the circumstance. These are some of many restrictions one faces when writing the desired behaviour in C++. Ultimately the code must be portable.

The performance of the logic operations come into question when constrained by a certain latency, in order for the block to operate as required. The clock period must stay below $1/f_{target}$, which in the case of the TPG core, the target frequency $f_{target} = 250$ MHz. This sets a limit of 4 ns on the clock period.

However, it must tick for long enough so that the block operations can occur within the desired latency. Performance comes at a trade to resource cost, as pipelining (parallelizing and rearranging operation initialisation to decrease the latency and input interval) employs more FPGA elements, which are a finite

[17] Pragmas are directives that are stored within the source code, rather than in an external `.tcl` file

[18] Software dedicated to simulating HDLs, typically used in collaboration with the Vivado project manager environment, where full VHDL block chains can be synthesized and implemented within an FPGA.

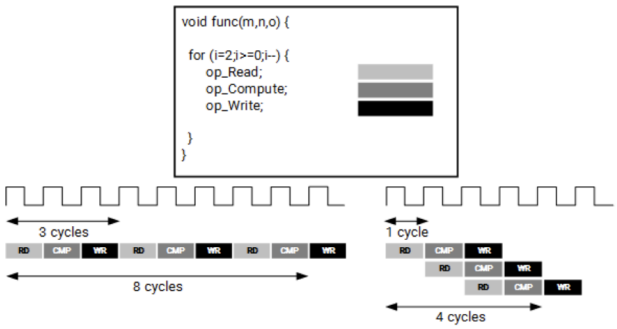


Figure 10: This figure contains an example showing how read, compute and write operations can happen in parallel to achieve higher throughput overall, using the `pipeline` directive.

resource. One way to reduce this cost is to assign accurate bit-widths to variables. Using standard C data types for hardware design results in unnecessary resource usage. Operations can use more LUTs and registers than needed for the required accuracy, causing delays that may exceed a given clock cycle and in turn increasing the overall latency. Smaller bit widths (e.g. using 17-bit data types for 17-bit multiplication rather than the standard C 32-bit data type) result in hardware operators which are smaller and faster. More logic can then fit in the FPGA which can operate at a higher clock frequency.

The next section will discuss the management of performance and area when choosing HLS design guidances.

2.2.1 Optimisations and Analysis

Directives act as a low-level guidance for the HLS tool, influencing the types of hardware used and the methods of implementing the source code operations [53]. A very useful example of this is the division of arrray elements into registers using the `array_partition` directive, allowing for faster data access at the cost of increased resources.

Different optimization directives can be applied before C synthesis; one of the most crucial is the `pipeline` directive. This instruction forces code tasks to execute non-sequentially, allowing the next execution of a task to begin before the current one finishes. This process is visualised in figure 10. One can infer from the figure that dependencies between two different operations can affect the software's capability to pipeline the entire function. This means the initiation interval[19] (II) cannot be minimised until code changes occur and dependency directives are implemented.

It can be deduced that if back pressure occurred during the operation of the pipelined function `func(m,n,o)`, then 3 clock cycles i.e. the number of active

[19]The number of clock cycles before new inputs are processed.

function 'layers' at any one time, would pass until the block is fully paused. This would prevent an instantaneous response to a signal, which is one of the core requirements for the designs discussed in section 3. Hence the need for an II equal to the latency. If we require a fully pipelined design, latency must equal one clock cycle.

In order to develop a fully pipelined block, code changes must be made to remove internal dependencies such as multiple store and load operations on the same registers and BRAMs (or any storage resource) at distinctly different times during the operation of the block. An example of a scenario that would cause a dependency issue is the writing of a register at the beginning of block operation and then a read from the same register towards the end. This kind of issue can be remedied by reducing the number of times to which a register is written via logic branching. This can then be further improved upon by applying `dependence` directives that identify false dependencies between arrays or variables, and allows the function operation to be maximally streamlined. The user must take caution with instructing HLS to recognise a false dependency for a variable where there could in fact exist a true dependency. This can lead to C/RTL co-simulation failure and block malfunction.

Additional functions called within the top function can employ the `inline` directive so as to reduce overhead and minimise pipelining issues when they are called. Not only can performance be adjusted by directives, but the HLS tool can also specify a limit to the resources used by the block, allowing for a balance between speed and area usage. Additionally, more customisation opportunities arise from the ability to select a block's IO protocol to make sure the final design can be correctly connected to other hardware blocks with the same framework.

There are several viable methods to achieve an optimal directive configuration for the user's purposes. One of the best ways to gauge the effect of adding directives in the Vivado HLS tool is by analysing the cycle-by-cycle display of operations within the synthesis reports. One can analyse a number of aspects surrounding the performance estimates of the block, perhaps the most important of which is clock timing. This is the fastest achievable clock frequency before the block ceases to function. Each design must meet this target to synchronise with the surrounding firmware blocks.

Secondly, the block latency and II must be reviewed, since the internal operations of the block must have enough time to run before the next input is processed. Without any pipeline directives, the latency defaults to one clock cycle less than the II so that this can be achieved. In many cases the top function logic and even the input data types must be rewritten several times before the design reaches its performance objectives. See section 3.1.5 for an example of this.

Finally, the utilization estimates are key to making the trade-off between latency and FPGA areas. Limited budget and gate area means that unlimited resource usage is not an option. Therefore these statistics must be tracked and adjusted for.

Bear in mind that some of the observations made in this section are specific to the design of signal filtration algorithms, and is not necessarily optimal for

all HLS design projects. There are many more directives and design techniques that can be found in the Xilinx documentation titled UG902 [48].

Now that a sufficient introduction into the design process has been given, the next section will explore the key work outlined in this report: the firmware algorithms employed by the HF block chain first mentioned in section 1.6.2.

3 Development and Findings

Of the three firmware blocks in the TPG core, the pedestal subtraction and FIR filter algorithms were successfully implemented in HLS. Sections 3.1 - 3.3 outline the operations performed by each algorithm as well as their performance and utilization statistics. Commentary will be made on certain tools and techniques that can be employed to circumvent common problems faced in HLS development. This should provide some insight into the relative quality of each redesigned block, and the accessibility of the HLS software.

3.1 Pedestal Subtraction Algorithm

The pedestal subtraction algorithm was the first mechanism in the project to be redesigned in HLS. It removes each channel's base background noise from the incoming event data, so that the following FIR and HF blocks are dealing with the isolated optical signals from the neutrino events occurring within the detector. This should not be confused with the reduction of random noise of the isolated signal, which is taken care of later in the block chain. This algorithm is implemented within the first sub-block in the TPG core, `pedsub`, which is fed 64 sample words[20] by the HSC shown in figure 8. Before the performance and resource utilization can be compared to the original design, it is necessary to demonstrate the algorithm's mechanisms to provide insight into the efficacy of HLS when implementing certain operations.

3.1.1 Algorithm Operations

The input packet initially contains 70 sample words. However, the first six words are stripped from the input by the HSC. The remaining samples are sent to the `pedsub` block. There are three important variables used to achieve the reduction of background noise: the pedestal (or median), the accumulator, and the incoming ADC sample. The pedestal represents the average background signal for a given data channel, of which there are 64 processed by each TPG unit. The pedestal is subtracted from each ADC value to produce the 'true' isolated output signal. The pedestal and accumulator are adjustable values that must be set to an estimate before any samples have entered the block. For the first 64 packets, the pedestal and accumulator are set to 500 and 0 respectively

[20] Each sample contains a 12-bit ADC value and 4 appended bits representing the associated AXI4 signals.

at the beginning of each packet. The reason for this will be discussed in section 3.1.2.

When the pedestal is greater than the incoming ADC value, the accumulator is decreased by one, and if the opposite is true, the accumulator is increased by one. When the accumulator reaches a limit of ± 10, the pedestal is increased by one or decreased by one respectively, and the accumulator is reset to zero. See figure 11 for an illustration of how the algorithm manages background signal noise.

Connected to this block, or implemented within it, is a state save and restore (SSR) mechanism, which is used to save and retrieve these adjustable variables at the end of a packet. For the HLS design that relies on an external SSR mechanism, this is implemented via a VHDL source file connected to the `pedsub` wrapper, that receives and feeds values to the pedestal subtraction logic. This can also be managed internally, the method for which is discussed in section 3.1.5. The following segment will describe how the SSR mechanism operates.

3.1.2 State Save and Restore

The SSR mechanism ensures the restoration of certain variables for a given data channel, once that channel has accepted at least one packet. Once all 64 data channels have received packets i.e. when the block processes its 65th packet, the SSR memory will have saved values associated with all channels. From this point onwards, each time a new packet enters a given channel, certain variables will be assigned values from the SSR memory element associated with that specific channel.

In the case of the pedestal subtraction algorithm processing channel N, the accumulator and pedestal values associated with this channel are saved at the end of a packet. If values were already saved for channel N, they have now been overwritten. In the same clock cycle, the values from channel N+1 are restored before the next packet is processed.

The reason for this is that packets are processed by each channel sequentially, beginning from channel 0 and reaching channel 63. For example, when `pedsub` finishes processing the packet and saving the state values associated with channel 5, the algorithm will then process packets entering channels $6 - 63$ and then channels $0 - 4$, after which the saved state values for channel 5 will be restored for further processing. This enables the generation of fine-tuned pedestal and accumulator values for the remainder of the detector run time. Consequently, one would obtain a better background noise estimate and therefore increasingly accurate ADC output data from the pedestal subtraction block.

3.1.3 Redesign of Pedestal Subtraction with an External SSR

The development and debugging of the `pedsub_HLS` block led to a number of insights for future use of high-level synthesis when designing detector firmware. The first thing to note is that when developing an algorithm, it is advisable to start with a simplified version of the target function. This is because complex

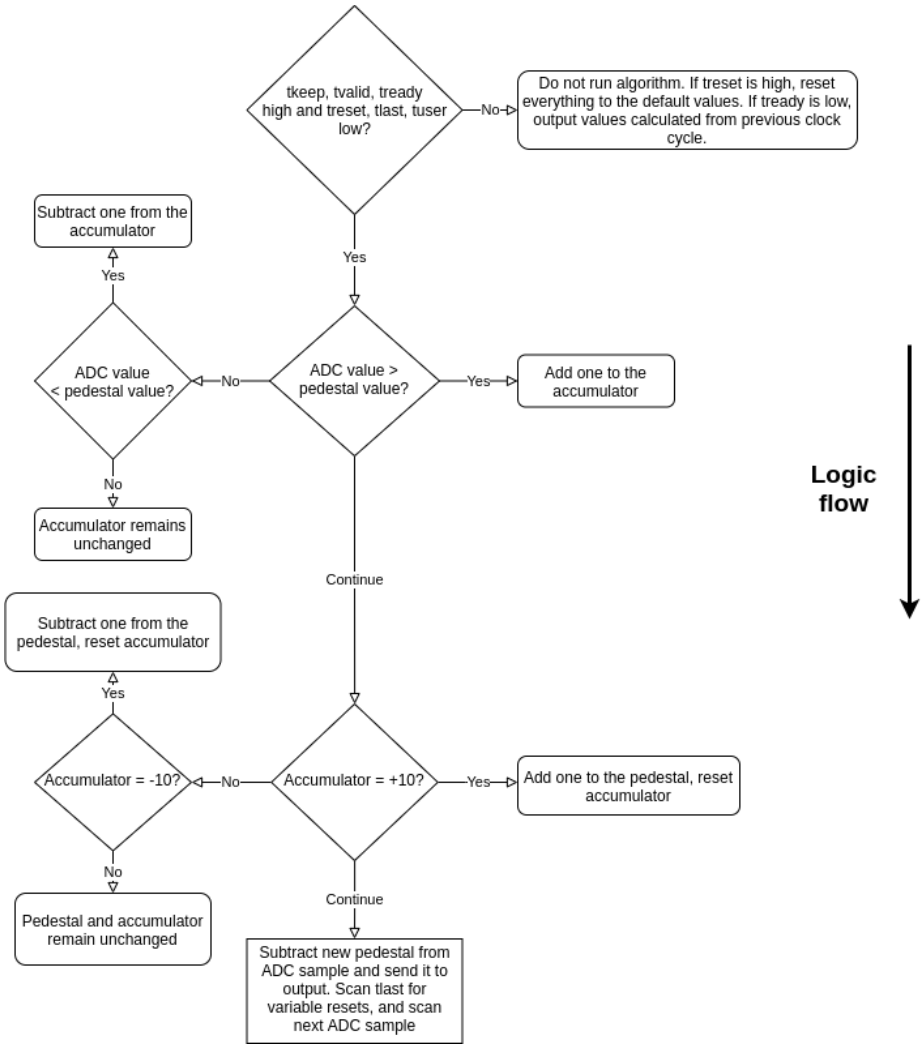


Figure 11: Flow chart detailing the logic flow of the pedestal subtraction algorithm, demonstrating the adjustment of the pedestal and accumulator values, as well as the subtraction from the ADC input.

functions are especially prone to invalid signals and incomplete output, which can be difficult to debug when trying to trace back the unfamiliar proxy signals in machine-written VHDL code.

More developmental success arose from fixing output issues within the HLS environment rather than trying to treat 'symptoms' of the problem by directly editing the IP block file. The user found difficulty in the debugging process due to limited and obscure Questasim warnings when testing HLS IP in a HDL simulation project. The output of RTL in the HLS environment often does not match the Questasim simulation results, creating obstacles in the design process. Observations can be made from the output signals displayed in the Questasim waveform, but frequently trial and error must be employed in order to target the source of an issue.

This is a drawback of HLS IP, since the post-synthesis design lacks transparency when compared to writing HDL projects from scratch. One of the best approaches to deal with this is to slowly add layers of complexity. Each time an addition to the code is implemented, one can test the functionality in Questasim and observe successful synthesis in the Vivado project manager before moving on.

Contingencies must be included within the wrapper to account for a disparity between the IP block and wrapper port's bit widths. Additional block logic may also be required to compensate for issues involving the timing of handshake signal outputs, such as with the delayed internal variable restoration when `tlast` goes high. This links into the inclusion of static variables that save internal values between function calls.

3.1.4 Static Variables and AXI4 Signal Behaviour

The Vivado HLS synthesis tool recognises a static variable in the same way a C++ compiler would, generating a static storage area for that value. An example of their use in the pedestal subtraction algorithm is the storage of a clock cycle counter that starts when the output `tlast` signal goes high. This static variable is saved between multiple function calls until it reaches three cycles. When this happens, the statically-stored pedestal and accumulator registers are restored to their SSR values for the next channel. Static variables are a crucial tool when a self-sufficient block is required and externalised IO management blocks are inconvenient.

However, one main issue faced earlier in the project was the long-term storage of the pedestal and accumulator. Static variables were automatically wiped between incoming packets. It can be seen from the `pedsub_HLS` Questasim waveform that this variable wipe occurs five clock cycles after the following packet's `tvalid` signal goes high.

A separate SSR storage array mechanism had to be developed for restoring values to a given channel in the long term[21]. In the preliminary report, HLS was deemed inconvenient when developing a long-term SSR solution; the block

[21] i.e saving and then restoring packets 64 packets later, or a minimum of 4480 clocks after the initial packet enters a channel

being called every clock cycle meant that variable definitions were reset very frequently, unless they were static. Further investigation needed to be made into the optimal approach, which was eventually found and utilised in the designs featured in sections 3.1.5 and 3.2.4.

Returning to the `tlast` mechanism, it was discerned that the automatic static variable wipe happens too late; the restoration of values 5 clock cycles after `tvalid` goes high for packet N means that the first 5 subtractions occur using the pedestal from packet $N-1$. This behaviour is not desired, and leads to 5 invalid outputs associated with the packet N. To deal with this, a manual restoration of the internal values must occur before `tvalid` goes high for packet N. However, this restoration must be after the `tlast` output goes high for packet $N-1$. The `tlast` input is not used as this would invalidate the pedestal subtraction of the 64th sample in packet $N-1$.

Analysing the Questasim waveform led to the realisation that restoring 3 clock cycles after the tlast output goes high allows for the pedestal and accumulator SSR restoration to correctly occur. This also satisfied the above constraints. This is an example of how chronology can affect the output of a block in unpredictable ways.

Another clock timing concern is the required instant response to back pressure signal `tready`. Traditionally, the block would react too late as a result of the latency delay from input to output. This issue can be circumvented in the case that the block takes a maximum of one clock cycle to complete.

The algorithm's first logic branch depends on the `tready` input being high or low. If `tready` is low i.e. back pressure is active and the block operation requires pausing, the mechanism will continuously force an output of the same values that were sent through one clock cycle before the back pressure went active. This will cease only when tready goes high again. The desired behaviour can be achieved by constantly saving the previous clock's register values, and then driving them to the output ports once the `tready` input goes low. However, if the block latency was greater than 1 clock cycle and the $\mathrm{II} = 1$, back pressure would occur halfway through the operation of one or more function calls, meaning the block would not react in time and erroneous outputs would be sent through to the FIR. This is the reason that all functional designs mentioned in this report have a latency of one clock cycle.

The final timing concern is ensuring that the outputs are transmitted at the same pace that the inputs are received, which can be achieved with the `pipeline` directive discussed in section 2.2.1 as well as careful optimisation of the logic.

The next section will discuss the disparity in resource cost between the original design and the new HLS design.

Performance and Resource Utilization Comparison

The HLS block `pedsub_HLS` exhibited half the latency of the original pedestal subtraction design. The synthesis report shows the block has a clock cycle of 3.6 ± 0.5 nanoseconds, and an RTL co-simulation confirmed the block satisfies

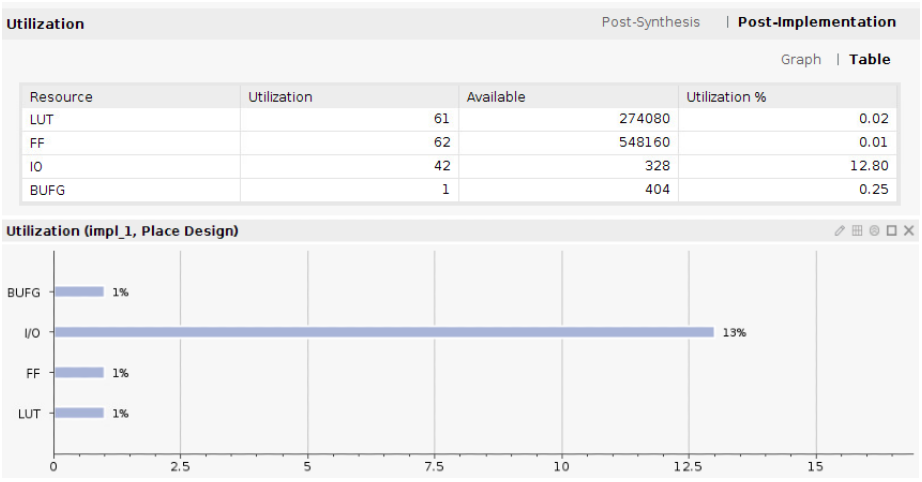


Figure 12: The resource utilization for the manually-developed pedestal subtraction block, including the exact figures above and percentage utilization below. All resource statistics were taken from the Vivado Project Manager post-implementation report summary, generated for each block. Please note that for all resource bar charts, utilization percentages above 1% have been rounded to the nearest percent. Utilizations below 1% are rounded up to the nearest percent.

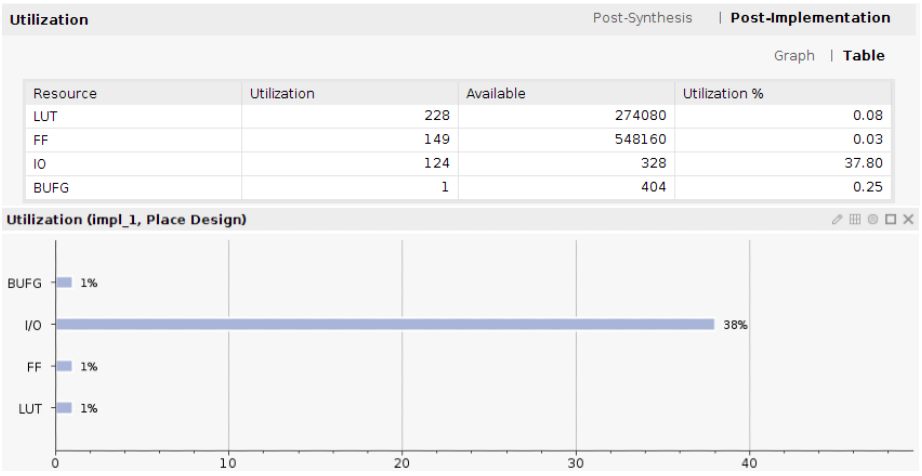


Figure 13: The resource utilization of the new pedestal subtraction HLS block connected to an external SSR, including exact figures in the statistics table above the graphical representation of the percentage utilizations.

the target frequency of 250 MHz. The HLS block outputs the same values as the previous design, ahead by one clock cycle due to the difference in latency. This was verified by a ModelSim simulation comparison between the blocks, and several model software output files.

When comparing resource utilization between the `pedsub` and `pedsub_HLS` blocks detailed in figures 12 and 13, the most important components to consider are LUTs and FFs. The input and output (IO)[22] resources for the FPGA target[23] and the BUFG[24] should not be taken into consideration when comparing resources.

The utilization tables show that the new design requires 0.06% more LUT usage when compared to the total LUTs available, which although comes to a 274% increase in total LUT usage when compared to the previous algorithm, is negligible when compared to the total available. Similarly, the FF usage for the HLS design shows an 140% increase from the original, but only a 0.02% difference in total usage of all available FFs. The disparity in LUT and FF usage is contributed to by the additional internal logic for the delayed SSR restoration and instantaneous back pressure mechanisms. An obvious third contributing factor is the increased performance of the HLS block.

Resource utilization could be reduced by further investigation into the AXI4 Stream Protocol directives described in section 3.2.2. Further savings could be made by removing unnecessary ports and incorporating multiple ports as members of a C++ `struct` type input. This method is used in the designs featured in section 3.1.5, 3.2.3 and 3.2.4.

Regardless of the efficiency observed in the HLS design, it is expected that the manual HDL method would have more opportunities for resource-efficient implementation when compared to the automated HLS process. This is the case for all designs featured in this report. The following segment will include discussion on the pedestal subtraction HLS block with in-built SSR behaviour.

3.1.5 Pedestal Subtraction with Implemented SSR

The pedestal subtraction algorithm `pedsub_HLS_SSR` is a HLS synthesized block including a memory-based SSR mechanism. The basic processing of the accumulator, pedestal and ADC values occur in exactly the same way as the design in section 3.1.3. However, to accommodate the performance deficit that came with storing and loading from a memory, the logic branching had to be adjusted. Additional temporary registers had to be utilised to avoid pipeline dependency issues. Arbitrary bit-width data types were used to reduce resource usage, and remove the need to truncate or zero-pad input or output signals in the wrapper. Sub functions called within the top function were inlined and pipelined to reduce overhead. C++ `struct` data types were used as the function arguments (i.e.

[22] These are physical structures that allow the connection of an FPGA target to other devices within the system.

[23] A programmable area or chip on the FPGA used to implement a digital circuit, in our case designed via HLS or directly through VHDL.

[24] A clock buffer.

the block ports) with the original AXI4 stream signals as the struct members.

The following section will explore the memory framework used to generate the required SSR behaviour in self-sufficient blocks.

Memory Utilization for the SSR Mechanism

The biggest change between `pedsub_HLS` and `pedsub_HLS_SSR` is the use of two global C++ arrays, defined outside the bounds of the top function, to save and restore the pedestal and accumulator values. These arrays can be accessed by the top function and act as permanent storage entities during runtime, solving the SSR problem faced in the preliminary report. HLS automatically implements these entities as two BRAM components. The drawback to BRAMs is that they support a very limited data access rate. This can cause problems during synthesis of the RTL files and needs to be worked around in some cases. The maximum access rate that a BRAM can support in this case is twice per clock cycle.

Fortunately, the only time `pedsub_HLS_SSR` needs to access each memory is to read and write once during the `tlast` restoration period. Therefore no pipelining or memory access issues were met at synthesis; the saving and restoring to the memory occurred seamlessly. However, in order to achieve a latency of one clock cycle, these arrays had to be fully partitioned into registers to increase access speed. These registers were generated as 'power-on initialisations' similarly to static variables, except the partitioned global arrays are not wiped between packets. For the `fir_HLS_SSR` block discussed section 3.2.4, the SSR memory was too large to be partitioned, and so it was necessary to employ BRAMs for the storing of SSR values. This avoided massive LUT and FF usage from a full array partition. During the project it was found that different conditions call for varying memory implementations.

The `pedsub_HLS_SSR` function had to be streamlined extensively in order to fit all operations into one clock cycle. In order to correctly save and restore the pedestal and accumulator, a static channel counter was used as a memory index between the values of 0 and 63. During the `tlast` restoration period at the end of each packet, the current channel's pedestal and accumulator values were saved to the SSR memory 2 clock cycles after `tlast` went high, and the next channel's values were restored on the following clock cycle. This avoided pipelining dependency issues involving the accessing of partitioned SSR arrays and the static accumulator and pedestal variables.

The variable reset functionality was removed in order to increase performance in compensation for the SSR. This reset mechanism was proven obsolete since all storage entities were automatically wiped by the external FELIX system at the beginning of a data run. The only issue encountered in this case was making sure non-zero starting values and booleans were set correctly at the start of a simulation, since these are improperly prepared by the automatic restart. This can be achieved by declaring and defining global entities as their starting value on the same script line, which are then changed after the first clock cycle.

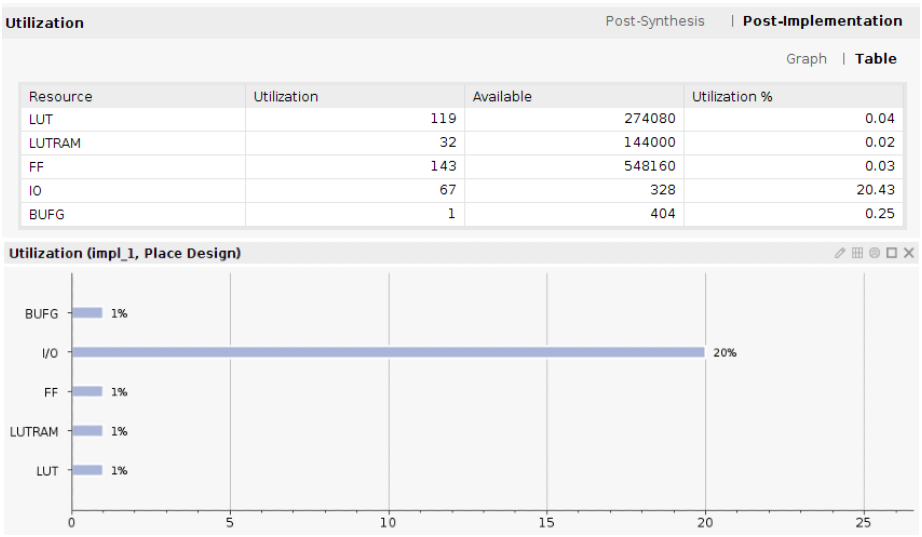


Figure 14: The resource statistics for the manual HDL pedestal subtraction algorithm, with the external VHDL SSR mechanism included in the statistics. This figure enables a comparison with the resources of the `pedsub_HLS_SSR` block

Performance and Resource Utilization Comparison

The main benefits of the redesigned pedestal subtraction algorithm with an in-built SSR are increased performance (faster throughput) and the convenience of a self-sufficient block. This means there are no external systems responsible for the SSR memory access behaviour. The clock period minimum of this design was shorter than the previous algorithm, reported at 3.0 ± 0.5ns. The simulation output for multiple packet waves with a randomly oscillating back pressure signal matches the model output files used to test the original pedestal subtraction SSR block. Based on the statistics featured in figures 14 and 15, comparisons can be made on the number of components required for each design.

The LUT usage of the HLS design is significantly higher at 481% of the original, or a non-negligible 0.26% usage increase of total LUTs available. This can be attributed to the LUTs required to process the operations on 128 registers for the pedestal and accumulator SSR arrays. FF usage increasing by 0.27% of the total amount available can also be attributed to the numerous SSR registers. Area usage can be dramatically reduced by un-partitioning the SSR memories and pursuing the methods described in section 3.2.2 so as to avoid the demanding requirement for latency = II = 1. This would use a small number of BRAMs instead of large numbers of LUTs and FFs, the analogue of which is seen in the original design requiring 32 LUTRAMs.

Section 3.2 will detail the operation and redesign of the FIR filter algorithm, before discussing the discoveries made during this process.

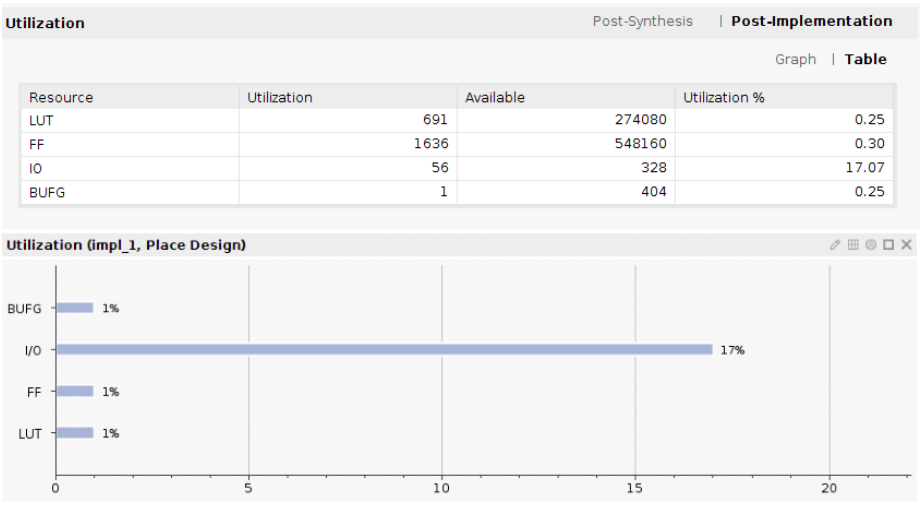


Figure 15: The resource statistics for the pedestal subtraction algorithm generated from HLS with an SSR mechanism built into the design. This is to be compared to the original pedestal subtraction block written directly in VHDL.

3.2 Finite Impulse Response Filter

The FIR algorithm acts as a random noise filter for the pedestal-subtracted ADC values, so that the final signal entering the HF block is clean enough to avoid generating false hits. Figure 16 illustrates the port connections between `pedsub_HLS_SSR` and `fir_HLS_SSR`[25]. Included in figure 17 is a visual representation of the effect that a low-pass, 32-tap filter has on the incoming ADC values. From this figure one can see how the hits can be more easily identified by the HF block, since the peaks are more distinct and the time spent above a certain threshold value can be more accurately measured.

3.2.1 Algorithm Operations

The FIR filter's core mechanism involves accepting the same number of samples per packet as the pedestal subtraction algorithm, but the difference is that each incoming ADC value is stored in one of 32 taps (dedicated registers). A visualisation of the FIR filter's operations can be seen in figure 18. A tap shift occurs every time a new input is processed, where the value at tap N moves to tap N+1. In other words, if we number the taps 0 to 31, and N new inputs have been processed since an input denoted I, then the input I will occupy tap N. Every clock cycle, input I will shift to tap $N+1$, then tap $N+2$, until it reaches the 31st tap. Input I is then forgotten on the next tap shift. Additionally, the

[25]The FIR filter IP.

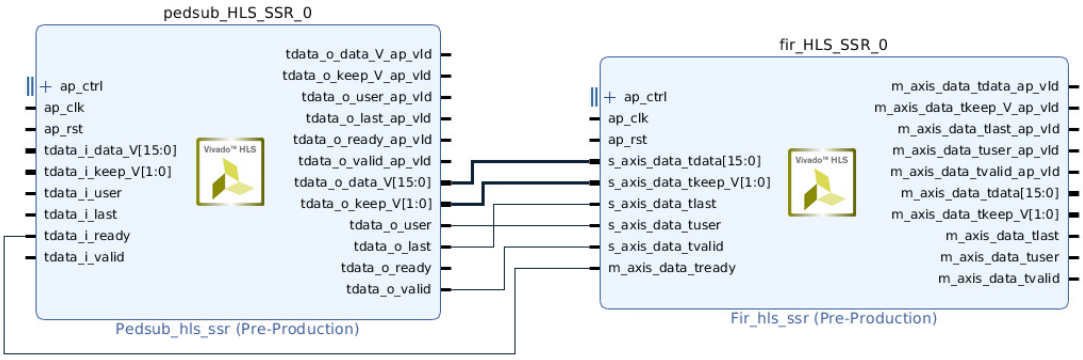


Figure 16: The block diagrams of both the pedestal subtraction and FIR filter HLS IP blocks, including the IO ports connections. This does not include the HSC interface on the left side of the pedestal subtraction block or the HF connections on the right side of the FIR filter block. `s_axis_data` and `tdata_i` denote left-side ports and `m_axis_data` and `tdata_o` denote right-side ports.

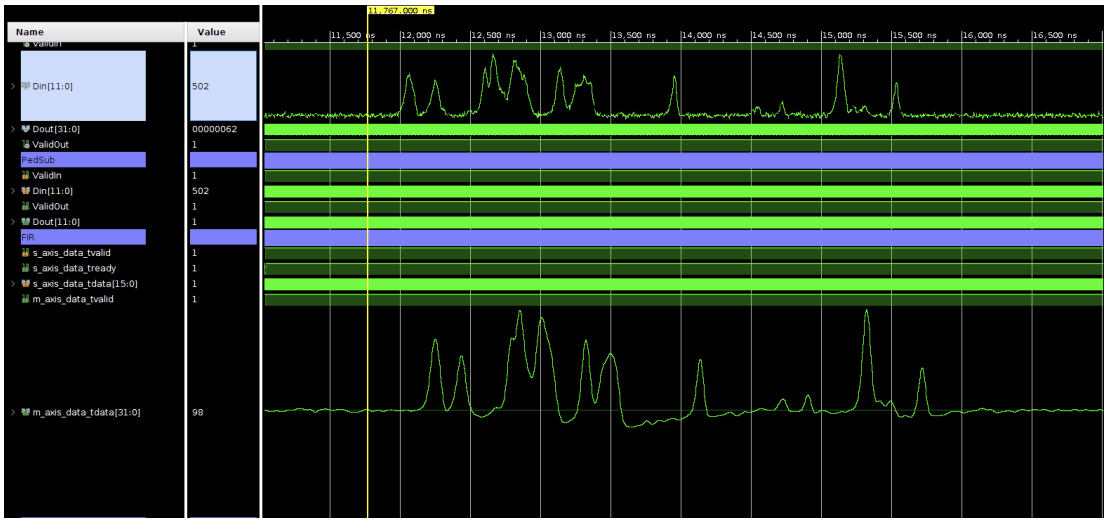


Figure 17: Analogue waveforms of the ADC input and output signals for the FIR filter block, generated early in the development of TPG core firmware. These are not to scale. The waveform above represents the pedestal-subtracted ADC outputs being fed into the FIR filter. These peaks are small in value and suffer from significant random noise. If this signal was fed into the HF algorithm, false hits would be much more likely. The signal below is the FIR filter output, with much larger peaks and much less random signal noise. These ADC values make it much easier to identify hits and generate less error.

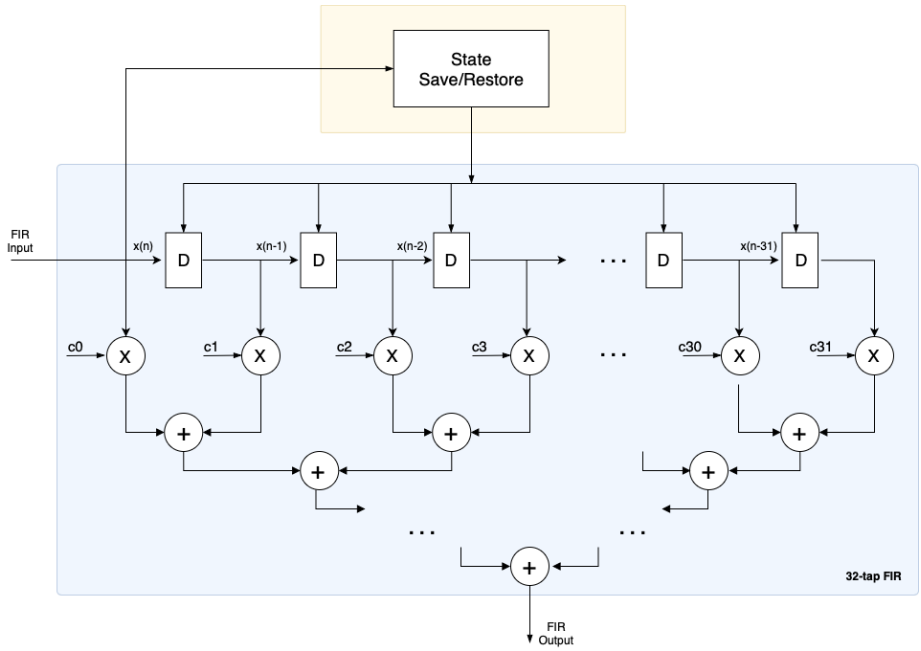


Figure 18: A visualisation of the FIR filter's manipulation of incoming ADC values, including the register shift, coefficient multiplication, summation tree and the saving and restoring of the final 32 tap values [45].

final 32 values of the packet are saved to an SSR memory array for the current channel. This is so the final 32 values can be restored to the tap registers next time the current channel receives a packet, similarly to the save and restore behaviour described in section 3.1.2.

The filter mechanism multiplies each tap by a certain customisable coefficient and sums the result of each multiplication together. This sum is sent to the ADC output. In figure 18 and in the manual HDL design, an addition tree is utilized to split the sums into 5 separate levels. The number of sums that occur is 16, 8, 4, 2 and 1 in that order. However, the approach taken in HLS was to sum each register individually in a loop, and then fully unroll that loop so all sums occur at the same time. This enabled the necessarily high performance but required significantly more resources.

Section 3.2.2 will discuss a possible workaround that would be investigated further if the project continued into the future.

3.2.2 Axis Directive Method for Back Pressure Behaviour

Initially, it proved difficult to achieve the performance required for the instantaneous back pressure response of a pipelined FIR block. An alternative solution was sought in order to obtain this behaviour for a block with latency greater

than one clock cycle. A workaround was found, and it involved using `axis` interface directives on the input and output ports of the design. This enabled HLS to automatically implement the AXI4 stream protocol behaviour. The most important aspect of this was the automatic, asynchronous back pressure from the HLS-generated `TVALID` and `TREADY` handshake detailed in Xilinx's AXI reference guide [54]. Side channel signals such as `TKEEP`, `TUSER` and `TLAST` could also be added as members of the `struct` ports.

After correcting the FIR wrapper to accommodate the new HLS AXI stream ports, Questasim simulations demonstrated working back pressure behaviour for blocks of latency greater than 1 clock cycle, when a simplified version of the design is tested. The issue with this approach is that HLS struggles to fully pipeline designs with `axis` port registers, which is a minimum requirement for all designs used in the TPG core. Additionally, a bug was found that hard-wired `TUSER` to high when the `TREADY` output signal was sent to the wrapper's output port. Previously there had been no `tready` output signal generated from the HLS blocks. The combination of these two issues led to pursuing the single-clock latency approach as a consequence of project time constraints. It remains to be said that if functional operation was achieved, the `axis` directive framework would have been a superior, generally-applicable solution for meeting the needs of FELIX's AXI specification system.

3.2.3 Redesign of FIR with External SSR

The HLS redesign of the original FIR block that relies on an external SSR is called `fir_HLS`, and when synthesized achieved a minimum clock period of 3.4 ± 0.5ns. It mimics the original algorithm, achieving the same output values for input files of 128 packets with random back pressure.

Mechanisms for `tlast` restoration, back pressure logic and struct ports were reused from `pedsub_HLS_SSR` for both this design and the following, discussed in section 3.2.4. Three sets of 32 tap registers were used, two sets of which were used to carry out high-performance computation of the tap shift and summation. The third set was employed for the restoration of SSR values received from the manually-developed SSR mechanism. `Constant` data type registers were partitioned and used to store the FIR coefficients, for maximum accessibility.

In both the HLS and manual HDL implementations, several of the coefficients were optimised by Vivado due to the fact that some of the current coefficients are zero. This also meant that less tap registers were required, as not all of them contributed to the summation. Only 23 were generated from the 32 C++ tap elements, yet valid output was still achieved in this configuration. An analogous example of this is the manual HDL version of the FIR filter requiring only 18 DSPs when implemented, as opposed to the expected 32 DSPs. This is also a consequence of the zero-value coefficients leaving room for optimisation.

The next segment will discuss the most expensive operation in the FIR filter, and how it was streamlined for maximum performance.

Tap Manipulation and Restoration

Two sets of tap registers were used in the core filtration mechanism. The reason for this was to increase the capability of HLS to parallelize the shifting and summation of the input ADC values. This is because internal loop dependencies occur when an array must read and write to itself on 32 different occasions, seen in the C++ code featured within listing 1.

Listing 1: The `for` loop responsible for the tap shift and summation in C++, using only one set of tap registers

```
for (short i = 31; i >= 0; i--) {

        // New ADC input introduced to first register
        if (i == 0) {
                tap_array[i] = ADC_input;
        }

        // Tap value shift
        else {
                tap_array[i] = tap_array[i-1];
        }
        // Sum of each tap occurs simultaneously
        sum += coefficients[i] * tap_array[i];
}
```

If this loop is modified so that there are two arrays that take turns being read from or written to i.e. `tap_array0[i] = tap_array1[i-1]` or vice versa, we can eliminate this dependency and achieve higher performance since all 32 sets of read and write operations can occur at the same time, including the summation. When the `tlast` signal goes high, the temporary SSR registers are restored to both tap arrays.

In order to correctly restore the state from the external SSR, additional related ports were implemented to accept and validate the incoming SSR tap values. The timing of the tap restoration is important; the SSR block outputs its first ADC value during the processing of the 33rd sample. Eventually, the SSR block outputs the final stored ADC value one clock cycle after the 64th sample has been processed. Contingencies were put in place to accommodate the misaligned output window. This is in contrast to the pedestal subtraction SSR output which restores both the median and accumulator values several clock cycles after the end of the packet, preventing any ambiguity in the restoration.

The next segment will demonstrate the resources required to implement the optimisations mentioned above.

Performance and Resource Utilization Comparison

The requirement for maximum performance of the `fir_HLS` design required many trial and error attempts at optimisation. Not only that, but it was necessary to parallelize the most time-expensive function operations: namely, the tap summation and SSR restoration. The significant disparity in the resources featured in figures 19-20 reflect the increased registers and associated operations required to create a functional FIR design in HLS whilst overcoming the obstacles created by FELIX's required AXI stream behaviour.

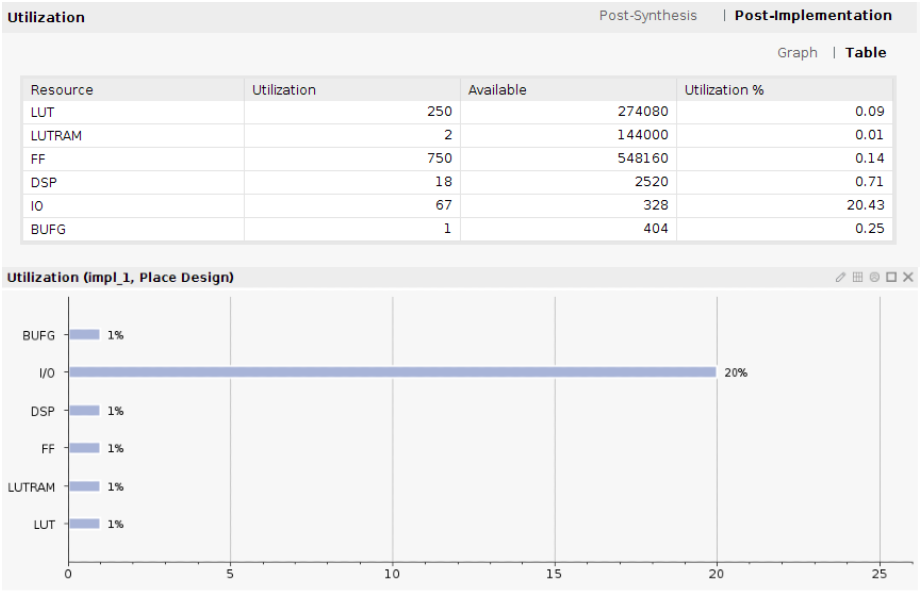


Figure 19: The resource statistics of the previously written FIR filter algorithm with the SSR mechanism not included, to allow for comparison with the analogous redesigned HLS module.

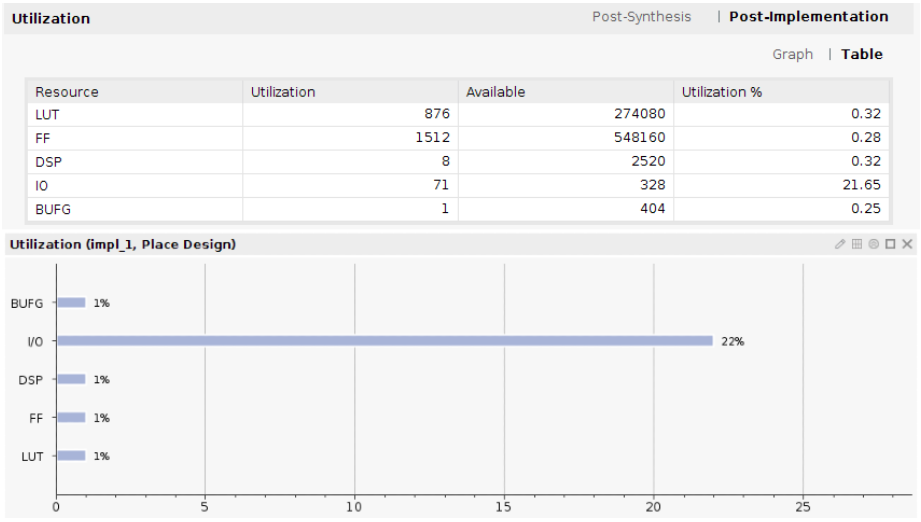


Figure 20: The resource statistics for the FIR filter algorithm written in HLS, connected to a previously written external SSR mechanism that is not included in the utilization. The only resources being spent on the SSR are those allocated for accepting and storing the output values from the external SSR block.

The LUT usage in the new HLS design is a 250% increase from the amount used by the previous block, with a non-negligible 0.23% increase in the total resource usage. The same goes for the number of FFs, with an 102% increase from the previous algorithm, and a notable 0.14% increase in total resource usage.

In contrast to with the pedestal subtraction algorithms, DSP usage is required in both the original and HLS blocks. However, the number of DSPs in the HDL block is 0.39% higher than the HLS design, in terms of the total available on the FPGA. HLS reduced this usage by employing optimisations drawn from the fact that certain FIR coefficients were zero and did not contribute to the summation. These optimisations required FFs and LUTS to implement. In the future, one could experiment with resource directives that instead force the employment of more DSPs to achieve the same task.

Section 3.2.4 will describe the FIR filter algorithm with a self-sufficient SSR mechanism.

3.2.4 FIR with Implemented SSR

The `fir_HLS_SSR` block employs a non-partitioned, two-dimensional array implemented as a BRAM, so that the design can be self sufficient when saving and restoring states. Tap restoration to the temporary SSR registers in preparation for the next channel occurs during the first 32 samples of each packet, i.e. one tap value restored per clock cycle. The reason for this is to avoid bottlenecking the read and write operations to the limited-access BRAM. This method ensures that there are no read operations occurring during the final 32 samples. This time period is reserved for the BRAM write operations, saving incoming ADC values so that they can be restored the next time that the current channel receives a packet.

In order for this block to reach the required performance, false dependency directives were applied to the temporary 32-register restoration array and the two-dimensional SSR array. This was a safe optimisation, since both entities were accessed in different branches of logic. Additionally, the same double-array tap shift method was used from section 3.2.3. This allowed full parallelization of SSR operations, and the design output was verified in the C/RTL and Questasim simulations. This meant that no dependence errors were generated since the RTL and C++ outputs matched.

Performance and Resource Utilization Comparison

The `fir_HLS_SSR` implementation has the biggest disparity in total resource area compared to the original design, likely due to the reasons discussed within section 3.2.3 in addition to the framework used for the internal SSR. This demanded the storing and retrieving of far more values than `pedsub_HLS_SSR`. Through the analysis of figures 21 and 22 it can be deduced that the HLS block requires 387% more LUTs than the manual HDL design, with a 0.36% increase in usage of total LUTs available (the largest yet). FF usage in the new block

Resource	Utilization	Available	Utilization %
LUT	261	274080	0.10
LUTRAM	2	144000	0.01
FF	747	548160	0.14
BRAM	1	912	0.11
DSP	18	2520	0.71
IO	42	328	12.80
BUFG	1	404	0.25

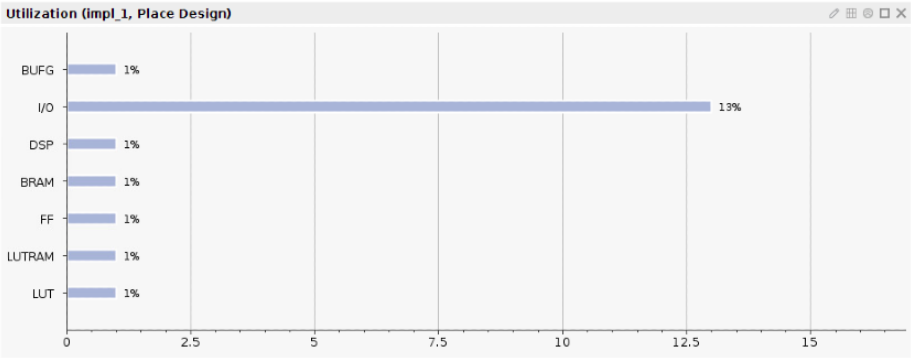


Figure 21: The resource statistics for the previously built FIR algorithm with the SSR mechanism included in the usage. This is included to allow the comparison with the HLS FIR block with an in-built SSR mechanism.

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	1272	274080	0.46
FF	1547	548160	0.28
BRAM	1	912	0.11
DSP	8	2520	0.32
IO	54	328	16.46
BUFG	1	404	0.25

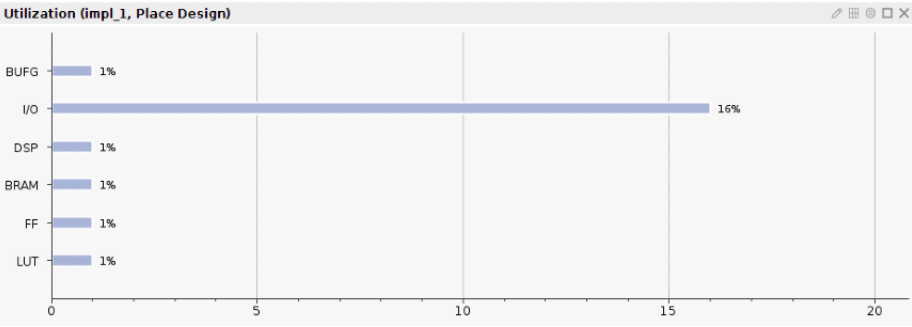


Figure 22: The resource statistics for the HLS-synthesized FIR algorithm with an in-built SSR memory and access mechanism. This enables a comparison with the manual HDL version.

came to a 107% increase from the previous design, again using 0.14% more of the total FFs available.

Statistics similar to those generated for `fir_HLS` emerge when looking at components that are not FFs and LUTs. An asymmetry that stands out between `fir_HLS` and `fir_HLS_SSR` is the use of one BRAM component for the in-built SSR memory, and the increased LUT usage that comes with processing 2048 BRAM elements.

3.3 Comparison Overview

The comparison of statistics in sections 3.1 - 3.2.4 provides insight into the behaviour most efficiently implemented by HLS when written by a relative beginner. The first and most simplistic design `pedsub_HLS` saw the most success. It achieved twice the operational speed and used a similar number of components to the manual HDL version. Subsequently it was found that introducing logical complexity to a maximum-performance design was especially demanding on the consumption of FPGA area. The disparity between HLS and manual HDL designs was largest for the FIR filter, due to the expensive operation of 32 tap shifts and summations occurring six times faster than the original algorithm. Not only that, but correctly accessing 2048 different SSR elements added a notable cost for the self-sufficient version.

Overall, huge resource savings would occur if each design could produce a valid output at a latency of greater than one clock cycle, but this was not achievable under the time constraints. Regardless, the fact that single-clock latencies were possible for all implementations is a testament to the power of HLS.

Section 4 will now condense and discuss the findings of section 3.

4 Design Process Conclusions

Now that the HLS and manual HDL algorithms have been compared, deductions can be made involving the debugging, performance and accessibility of HLS implementations. One of the steepest learning curves was correctly simulating IP blocks using Questasim, which is explained in section 4.1.

4.1 Simulation using Questasim

Arguably, Questasim is a very useful software for gauging the behaviour of HLS IP in a more realistic environment i.e. closer to the conditions the firmware would experience in a detector. However, the disparity between validated results in the HLS testbench environment and the complete malfunction of output signal drivers in the digital circuit can lead to roadblocks in the design process. Increased development productivity comes at a price. Once the IP is synthesized in HLS it proves difficult to analyse the machine-generated code.

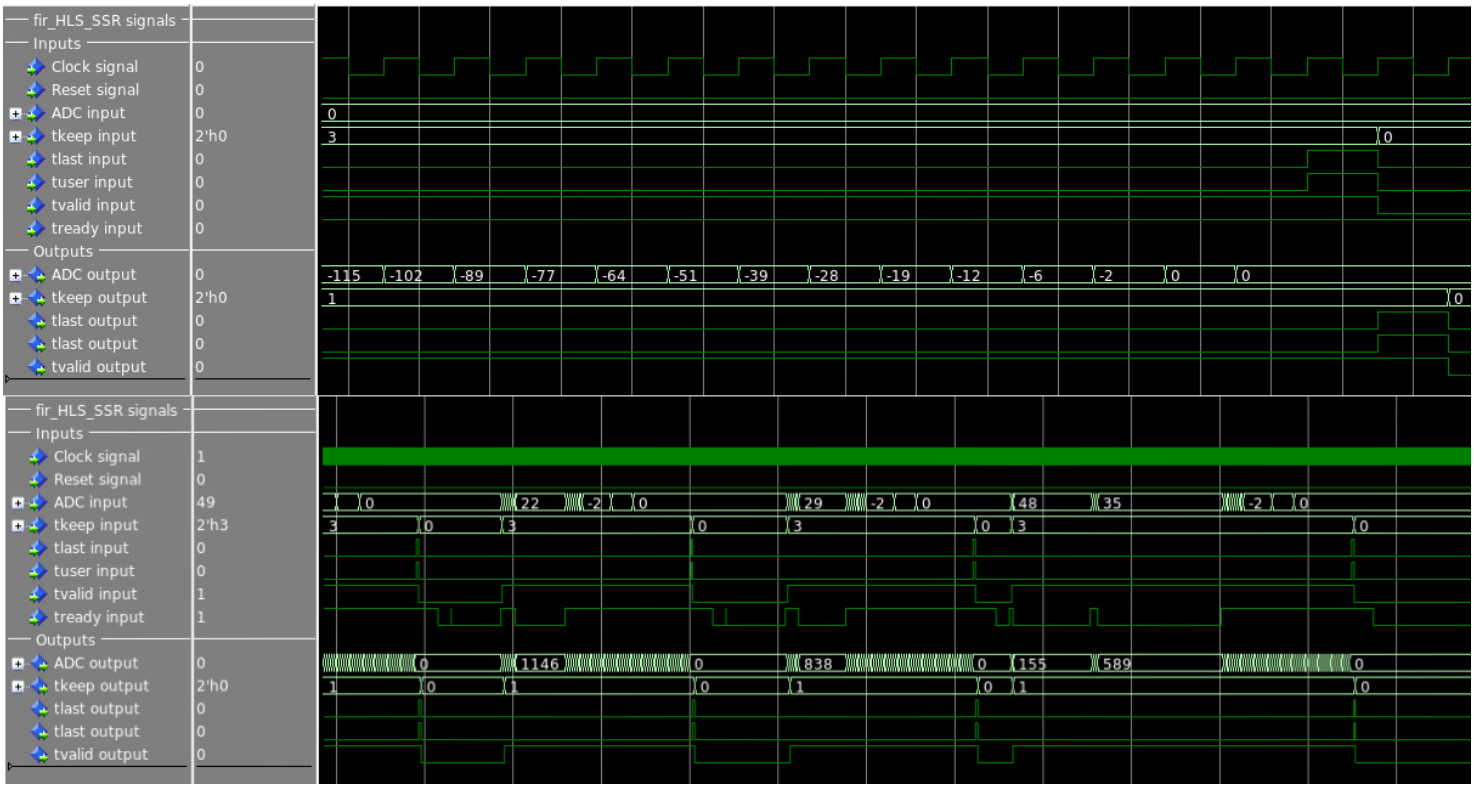


Figure 23: A slice of the waveform featuring all input and output port signals for `fir_HLS_SSR` in the Questasim simulation environment. The image above features a zoom-in view on signal behaviour at the end of a singular packet, and the image below visualises several packets. Note the accessible digital readout for ADC input and output, rapidly changing packet bunches during high `tvalid` and `tkeep` input signals, and no change in the ADC value after a high `tlast` input. The `tready` input signal oscillates randomly during the simulation, pausing block operations temporarily. The pause mechanism simulates sequential blocks sending back pressure due to limited throughput and overwhelming traffic. Questasim waveforms provide valuable information into the block operation and the clock timing of each AXI4 signal.

The Questasim waveform can create a window into the signals contributing to an improperly-driven or misbehaving output. Figure 23 provides a visualisation of the signal patterns for `pedsub_HLS_SSR`. Whilst testing the output in Questasim, it is possible to monitor the waveform of both the manual HDL and HLS implementations side-by-side for comparison purposes. Any disparity in signals would provide clues for why the output values differ from the validated results.

Timing-specific problems are best analysed in the signal waveform, where the user can identify misalignments of inputs and outputs. If the signal analysis approach is unsuccessful, then the remaining solution is to try a different method of implementing the desired behaviour. Another fall back option is the development of a simplified version of the malfunctioning algorithm in order to identify which specific operation generated issues. This proved to be time-consuming and imprecise.

There were several points during development where simulations froze inexplicably when processing a certain component of the IP, and were fixed by arbitrary changes in the wrapper. Partially pipelining of a design i.e. an average II = 1 but a maximum of II = 2 was one of several causes of said freezing. Other triggers of erroneous simulation runtime included mismanagement of the IP source hierarchy in the project configuration file, or the incorrect activation of certain blocks in the TPG chain. On occasion the output text file displayed values that did not correlate with the waveform, generating questions as to how Questasim visualised outputs versus their actual readout.

The customisability of manual HDL development would provide considerably more transparency into debugging when faced with the Questasim validation phase. However, HLS provides the edge in productivity during the previous design phases.

Section 4.2 will outline the experiences of the wider FPGA community.

4.2 General Consensus

Before conclusions can be made on the research featured in this report, it is important to note that this is not the first time HLS has been considered for a particle physics application.

One general conclusion of the community is that HLS produces a less efficient implementation, but allows for faster development time and better design productivity [55–57]. Other papers discuss the tools provided by Vivado HLS as mature enough to consider as the first approach for designing FPGA algorithms. HLS has even been reported to provide similar efficiency or even more efficient implementations [58, 59], especially when dealing with the re-use of certain resources [60].

An article titled 'Implementation of OMTF Trigger Algorithm with High-level Synthesis' [61] describes the advantages and disadvantages of HLS for the 'Overlap Muon Track Finder'. The main algorithm being studied was translated successfully from C++ into an FPGA implementation. Not only that, but the source code was more concise and easier to understand, maintain and test. HLS

ensures that data in parallel paths are synchronised in a pipelined design. This is not something one can take for granted when developing VHDL code manually.

One of the described drawbacks was that some of the tested algorithms were impossible to correctly translate from C++ into synthesizable HDL code. Additionally, for the user to properly prepare the C++ code, they may require prior knowledge of FPGA operation, which reduces accessibility. Complex designs took significantly more time to synthesize and implement in Vivado Project Manager, and the generated HDL code is bloated and illegible.

Another account is given in the report titled 'Software and Firmware Co-development Using High-level Synthesis' [62], featuring trigger applications from the CERN CMS experiment. For the purpose of bridging the gap between an initial prototype C++ model and the final FPGA algorithm, HLS was described to be exceptional in saving time between abstraction and implementation. All HLS modules featured in the CMS report satisfied their strict performance constraints and displayed better resource usage than the manual Verilog implementation. The HLS hardware output matched perfectly with the original results.

All of the above findings resonate with the discoveries made in this project. Many of the most popular opinions on HLS design are shared across the academic board. Whether the tools of HLS should be employed in an experimental context is best assessed on a case-by-case basis. The reason for this is that a user's success can vary dramatically depending on the behaviour that they require and their previous experience with FPGAs.

Given more time to investigate the `axis` interface directives detailed in section 3.2.2, similar insight could have been provided into the resource usage of pipelined HLS blocks with the same performance as the manual HDL designs. In this case, the efficiency of high-level synthesised implementations could be brought into question.

4.3 Conclusion

In this document, the theoretical and experimental motivations for the DUNE experiment have been outlined. Details of the HF architecture and the HF algorithms were discussed, and the original VHDL designs were compared to the new HLS implementations. The following segments summarise the discoveries made during the progression of the project. The section will conclude with a discussion of how the continuation of this work could be pursued.

4.3.1 Technical Findings

During the design of the firmware algorithms discussed in this report, once the user became more familiar with the HLS tools, design productivity increased dramatically. Functional IP blocks were implemented within increasingly shorter periods as time went on.

One of the biggest revelations during the early stages of the project was the use of static C++ variables. This allowed the block to internally save values

between function calls, which meant that no external mechanisms were required to maintain register continuity. The algorithm became self sufficient.

Externally defined C++ arrays were especially helpful when saving and restoring variable states between packets. These could be partitioned for performance or automatically implemented as BRAMs for low-cost memory storage. These arrays effectively circumvented any externally-driven block wipe between packets, as they were permanently defined.

Employing arbitrary bit widths using `ap_int.h` and the `ap_int<bitwidth>` datatype saved area usage, as predicted in section 2.2.1. Using the C++ `struct` datatype as a function argument led to the synthesis of port interfaces with drastically lower IO resource usage. A `struct` is the recommended port type when using `axis` interface directives with side channels like `tlast`, `tuser` and `tkeep`. This `axis` directive approach is the best way to generate designs that satisfy general AXI4 stream protocol requirements, such as an instantaneous response to back pressure. However, this approach is met with difficulty and requires further investigation to achieve a successful implementation.

It should be noted that the back pressure behaviour of the manually written IPs was intuitively implemented by a line of VHDL code that acted as a 'clock enable' to all block processes, pausing all operations when back pressure occurred. It can be inferred that AXI4 stream behaviour is far more implementable in manual HDL projects than in their HLS counterparts. This is especially the case for a beginner in FPGA development.

The high-performance approach to mimicking AXI4 stream behaviour was effective, and forced the user to streamline their designs much further than the manual HDL versions. It is worth mentioning that even complex functions can support high throughput via persistence and familiarity with the optimisation directives. For minimum block latency, the most useful directives were `pipeline`, `array_partition`, `dependence` and `inline`. However, maximum throughput led to larger area usage than was necessary, since fast block operation was not a priority in the FELIX system.

Overall, the component utilization of the HLS blocks were significantly larger than those written manually in VHDL, but not large enough to cause implementation issues. LUT and FF usage was especially higher in the HLS versions. Despite this, total resource costs consistently remained below 0.5% of the total available for the ZYNQ UltraScale+ FPGA [26].

4.3.2 The User Experience

The first thing one must realise when writing in a high-abstraction language for the purpose of FPGA implementation is that the style is vastly different to how software is usually developed. One must possess a basic level of knowledge regarding FPGAs and digital circuits, and realise that sometimes code must be written with clock chronology in mind. One of the purposes of HLS is to allow the user to avoid clock chronology contingencies via interface directives, but

[26] Part name xzcu9eg-ffvb1156-2-e.

this is not always possible and sometimes improvisations must be made. An example of this is the `tlast` restoration mechanism outlined in section 3.1.

Increased logic branching is encouraged, since it can often improve performance in FPGA firmware, in contrast to CPU software. Frequently the HLS top function requires more arguments and internal variables than usual, to ensure all handshake signals are accepted and correctly sent to the output. These are just a few of many examples that prove it is impossible to separate the C++ coding style from the limitations of HDLs.

Secondly, HLS development requires a consistent trial and error approach due to the illegibility of the synthesized code as mentioned in sections 4.1 and 4.2. This is a significant disadvantage when compared to the customisability of manual VHDL programming. Despite this drawback, and despite its tendency for less efficient resource implementation, HLS allows for higher design productivity.

High level synthesis enables physicists unfamiliar with HDLs to generate IP blocks. Originally it was only trained engineers, who are few and far between in the experimental community, that were able to achieve this. Detailed documentation and extensive tutorials on the Xilinx support page enable beginners to quickly familiarise themselves with the Vivado HLS GUI and console. This is in contrast to the much lengthier training periods for manual HDL development.

Consequently, the prospects for accelerating the development of firmware for ProtoDUNE II or any other high-performance experiment are enormous.

The work carried out for this report took place during the COVID-19 pandemic and required remote working for the majority of its duration. The user had no exposure or prior knowledge about FPGAs or the HLS software, yet the ease of the HLS design process enabled fast progression and notable success. This is a testament to the power of this method, and is a recommendable option for all software developers and physicists to consider.

4.3.3 Future Work

Despite the fact that most project goals were met, there were some aspects of the project that were left unfinished. The top priority for future work would involve the creation of a block whose interface behaviour is automatically handled by HLS. For the designs featured in section 3, this would be an asynchronous back pressure response for block latencies greater than or equal to one, in addition to the correct variable restoration at the end of a packet. This is so that clock timing can be exclusively handled by the software and no longer plays a role in design choices.

In the case of both the HLS pedestal subtraction and FIR filter algorithms, automatically generated AXI4 stream ports and associated behaviours would allow for a latency of 2 clock cycles and 6 clock cycles respectively. These latencies are shared by their analogous manual HDL blocks. Enabling sub-maximal performance in the HLS versions would dramatically reduce resource cost, and allow for a fair comparison between software and engineer implementations exhibiting the same performance.

It cannot be ignored that the third sub-block in the TPG block chain remains to be redesigned. This would add further insight into how HLS deals with various operational requirements. Following the successful implementation of all three HLS sub-blocks in the TPG core, one could connect their mutual IO ports and test the functionality and resource cost of a full TPG unit developed in HLS.

The last aspiration would be the generation of DUNE firmware that has not previously been developed in manual HDL. One could gauge the difficulty met when acquiring the correct output signals without the scaffolding of a 'blueprint' script. This could be compared to the redesign process. Conclusions could be made on whether HLS is a feasible tool when developing brand-new firmware for the DUNE experiment.

References

[1] "Dune science home page." `https://www.dunescience.org/`, 2020. Accessed 11-01-2021.

[2] D. Griffiths, "Introduction to elementary particles, john willey & sons," 1987.

[3] F. Abe, H. Akimoto, A. Akopian, M. G. Albrow, S. R. Amendolia, D. Amidei, J. Antos, C. Anway-Wiese, S. Aota, and G. e. a. Apollinari, "Observation of top quark production in $p^{-}p$ collisions with the collider detector at fermilab," *Physical Review Letters*, vol. 74, p. 2626–2631, Apr 1995.

[4] K. Kodama, N. Ushida, C. Andreopoulos, N. Saoulidou, G. Tzanakos, P. Yager, B. Baller, D. Boehnlein, W. Freeman, and B. e. a. Lundberg, "Observation of tau neutrino interactions," *Physics Letters B*, vol. 504, p. 218–224, Apr 2001.

[5] C. P. Off, "Cern experiments observe particle consistent with long-sought higgs boson," 2012.

[6] F. R. KLINKHAMER, "Neutrino mass and the standard model," *Modern Physics Letters A*, vol. 28, p. 1350010, Feb 2013.

[7] T. Ohlsson, "Special Issue on 'Neutrino Oscillations: Celebrating the Nobel Prize in Physics 2015' in Nuclear Physics B," *Nuclear Physics B*, vol. 908, p. 1, July 2016.

[8] "Atmospheric neutrinos: Super-kamiokande official website." `http://www-sk.icrr.u-tokyo.ac.jp/sk/sk/atmos-e.html`.

[9] S. Mertens, "Direct neutrino mass experiments," *Journal of Physics: Conference Series*, vol. 718, p. 022013, May 2016.

[10] P. S. Cooper, "The masses of the neutrinos," *arXiv preprint arXiv:1605.03159*, 2016.

[11] R. Acciarri, M. A. Acero, M. Adamowski, C. Adams, P. Adamson, S. Adhikari, *et al.*, "Long-baseline neutrino facility (lbnf) and deep underground neutrino experiment (dune) conceptual design report volume 1: The lbnf and dune projects," 2016.

[12] S. M. Bilenky, "Neutrinos: Majorana or dirac?," *Universe*, vol. 6, no. 9, p. 134, 2020.

[13] C.-S. Wu, E. Ambler, R. W. Hayward, D. D. Hoppes, and R. P. Hudson, "Experimental test of parity conservation in beta decay," *Physical review*, vol. 105, no. 4, p. 1413, 1957.

[14] S. King, "Unified models of neutrinos, flavour and cp violation," *Progress in Particle and Nuclear Physics*, vol. 94, pp. 217–256, 2017.

[15] W. Rodejohann, "Neutrino-less double beta decay and particle physics," *International Journal of Modern Physics E*, vol. 20, no. 09, pp. 1833–1930, 2011.

[16] B. Schwingenheuer, "Status and prospects of searches for neutrinoless double beta decay," *Annalen der Physik*, vol. 525, no. 4, pp. 269–280, 2013.

[17] W. Xu, N. Abgrall, F. T. Avignone, A. S. Barabash, F. E. Bertrand, V. Brudanin, M. Busch, M. Buuck, D. Byram, and A. S. e. a. Caldwell, "The majorana demonstrator: A search for neutrinoless double-beta decay of 76ge," *Journal of Physics: Conference Series*, vol. 606, p. 012004, May 2015.

[18] N. Khanbekov, "Amore: Collaboration for searches for the neutrinoless double-beta decay of the isotope of 100 mo with the aid of 40 ca 100 moo 4 as a cryogenic scintillation detector," *Physics of Atomic Nuclei*, vol. 76, no. 9, pp. 1086–1089, 2013.

[19] J. B. Albert, G. Anton, I. J. Arnquist, I. Badhrees, P. Barbeau, D. Beck, V. Belov, F. Bourque, J. P. Brodsky, and E. e. a. Brown, "Sensitivity and discovery potential of the proposed nexo experiment to neutrinoless double-beta decay," *Physical Review C*, vol. 97, Jun 2018.

[20] A. D. Sakharov, "Violation of cp-invariance, c-asymmetry, and baryon asymmetry of the universe," in *In The Intermissions... Collected Works on Research into the Essentials of Theoretical Physics in Russian Federal Nuclear Center, Arzamas-16*, pp. 84–87, World Scientific, 1998.

[21] A. D. Dolgov, "Baryogenesis, 30 years after," *Surveys in High Energy Physics*, vol. 13, p. 83–117, Nov 1998.

[22] W. Rodejohann, "Neutrino mixing and neutrino telescopes," *Journal of Cosmology and Astroparticle Physics*, vol. 2007, no. 01, p. 029, 2007.

[23] A. G. Cohen, S. L. Glashow, and Z. Ligeti, "Disentangling neutrino oscillations," *Physics Letters B*, vol. 678, p. 191–196, Jul 2009.

[24] M. Aartsen, M. Ackermann, J. Adams, J. Aguilar, M. Ahlers, M. Ahrens, I. Al Samarai, D. Altmann, K. Andeen, T. Anderson, *et al.*, "Search for nonstandard neutrino interactions with icecube deepcore," *Physical Review D*, vol. 97, no. 7, p. 072009, 2018.

[25] J. Schechter and J. W. Valle, "Neutrino masses in su (2) $\bigotimes$ u (1) theories," *Physical Review D*, vol. 22, no. 9, p. 2227, 1980.

[26] A. Boyarsky, M. Drewes, T. Lasserre, S. Mertens, and O. Ruchayskiy, "Sterile neutrino dark matter," *Progress in Particle and Nuclear Physics*, vol. 104, pp. 1–45, 2019.

[27] T. Yanagida, "Horizontal symmetry and masses of neutrinos," *Progress of Theoretical Physics*, vol. 64, no. 3, pp. 1103–1105, 1980.

[28] R. Sachs, "Cp violation in k 0 decays," *Physical Review Letters*, vol. 13, no. 8, p. 286, 1964.

[29] M. Kobayashi and T. Maskawa, "Cp-violation in the renormalizable theory of weak interaction," *Progress of theoretical physics*, vol. 49, no. 2, pp. 652–657, 1973.

[30] L. Wolfenstein, "Parametrization of the kobayashi-maskawa matrix," *Physical Review Letters*, vol. 51, no. 21, p. 1945, 1983.

[31] P. D. Group, P. Zyla, R. Barnett, J. Beringer, O. Dahl, D. Dwyer, D. Groom, C.-J. Lin, K. Lugovsky, E. Pianori, *et al.*, "Review of particle physics," *Progress of Theoretical and Experimental Physics*, vol. 2020, no. 8, p. 083C01, 2020.

[32] P. Di Bari, "An introduction to leptogenesis and neutrino properties," *Contemporary Physics*, vol. 53, p. 315–338, Jul 2012.

[33] V. A. Kuzmin, V. A. Rubakov, and M. E. Shaposhnikov, "On anomalous electroweak baryon-number non-conservation in the early universe," *Physics Letters B*, vol. 155, no. 1-2, pp. 36–42, 1985.

[34] B. Kaysor, "The search for leptonic cp violation." `https://cerncourier.com/a/the-search-for-leptonic-cp-violation/`, Aug 2020.

[35] Z. Maki, M. Nakagawa, and S. Sakata, "Remarks on the unified model of elementary particles," *Progress of Theoretical Physics*, vol. 28, no. 5, pp. 870–880, 1962.

[36] B. Abi, R. Acciarri, M. A. Acero, G. Adamov, D. Adams, M. Adinolfi, Z. Ahmad, J. Ahmed, T. Alion, S. A. Monsalve, *et al.*, "Deep underground neutrino experiment (dune), far detector technical design report, volume ii dune physics," *arXiv preprint arXiv:2002.03005*, 2020.

[37] B. Abi, R. Acciarri, M. Acero, G. Adamov, D. Adams, M. Adinolfi, Z. Ahmad, J. Ahmed, T. Alion, S. A. Monsalve, *et al.*, "Long-baseline neutrino oscillation physics potential of the dune experiment," *The European Physical Journal C*, vol. 80, no. 10, pp. 1–34, 2020.

[38] B. Abi, R. Acciarri, M. A. Acero, G. Adamov, D. Adams, M. Adinolfi, Z. Ahmad, J. Ahmed, T. Alion, S. A. Monsalve, *et al.*, "Volume i. introduction to dune," *Journal of Instrumentation*, vol. 15, no. 08, p. T08008, 2020.

[39] B. Abi, G. Barr, J. Martin-Albo, and F. Azfar, "Dune's daq architecture; envisaging the alternatives," Aug 2017.

[40] T. Collaboration, B. Abi, R. Acciarri, M. Acero, M. Adamowski, C. Adams, D. Adams, P. Adamson, M. Adinolfi, Z. Ahmad, C. Albright, T. Alion, J. Anderson, K. Anderson, C. Andreopoulos, M. Andrews, R. Andrews, J. Anjos, A. Ankowski, and R. Zwaska, "The single-phase protodune technical design report," 06 2017.

[41] C. Rubbia, "The liquid-argon time projection chamber: a new concept for neutrino detectors," tech. rep., 1977.

[42] C. Adams, M. Alrashed, R. An, J. Anthony, J. Asaadi, A. Ashkenazi, M. Auger, S. Balasubramanian, B. Baller, C. Barnes, *et al.*, "Rejecting cosmic background for exclusive neutrino interaction studies with liquid argon tpcs; a case study with the microboone detector," *arXiv preprint arXiv:1812.05679*, 2018.

[43] B. Aimard, C. Alt, J. Asaadi, M. Auger, V. Aushev, D. Autiero, M. Badoi, A. Balaceanu, G. Balik, L. Balleyguier, *et al.*, "A 4 tonne demonstrator for large-scale dual-phase liquid argon time projection chambers," *Journal of Instrumentation*, vol. 13, no. 11, p. P11003, 2018.

[44] A. Borga, E. Church, F. Filthaut, E. Gamberini, R. Habraken, G. L. Miotto, T. Miedema, F. Schreuder, J. Schumacher, R. Sipos, *et al.*, "Felix based readout of the single-phase protodune detector," in *EPJ Web of Conferences*, vol. 214, p. 01013, EDP Sciences, 2019.

[45] K. Manolopoulos, "Trigger primitive generation - firmware." `https://indico.fnal.gov/event/44240/contributions/190328/attachments/132052/162111/UPDAQ_TechRev_TPGfw.pdf`, Jul 2020.

[46] D. Adams, M. Bass, M. Bishai, C. Bromberg, J. Calcutt, H. Chen, J. Fried, I. Furic, S. Gao, D. Gastler, *et al.*, "The protodune-sp lartpc electronics production, commissioning, and performance," *Journal of Instrumentation*, vol. 15, no. 06, p. P06017, 2020.

[47] P. Coussy, D. D. Gajski, M. Meredith, and A. Takach, "An introduction to high-level synthesis," *IEEE Design & Test of Computers*, vol. 26, no. 4, pp. 8–17, 2009.

[48] "Vivado design suite user guide 2018." `https://www.xilinx.com/support/documentation/sw_manuals/xilinx2017_4/ug902-vivado-high-level-synthesis.pdf`, Dec 2018. Xilinx.

[49] F. Vahid, *Digital design with RTL design, VHDL, and Verilog*. John Wiley & Sons, 2010.

[50] S. K. Bose, *Hardware and Software of Personal Computers*. New Age International, 1996.

[51] M. Hall and J. P. McNamee, "Improving software performance with automatic memoization," *Johns Hopkins APL Technical Digest*, vol. 18, no. 2, p. 255, 1997.

[52] B. G. Liptak, *Process Control and Optimization. Instrument Engineers' Handbook, Volume II.* CRC Press, 2006.

[53] C. Lo and P. Chow, "Model-based optimization of high level synthesis directives," in *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*, pp. 1–10, IEEE, 2016.

[54] "Xilinx axi reference guide (ug1037)." `https://www.xilinx.com/support/documentation/ip_documentation/axi_ref_guide/latest/ug1037-vivado-axi-reference-guide.pdf`, Jul 2017.

[55] T. Marc-André, "Two fpga case studies comparing high level synthesis and manual hdl for hep applications," *arXiv preprint arXiv:1806.10672*, 2018.

[56] M. Pelcat, C. Bourrasset, L. Maggiani, and F. Berry, "Design productivity of a high level synthesis compiler versus hdl," in *2016 International Conference on Embedded Computer Systems: Architectures, Modeling and Simulation (SAMOS)*, pp. 140–147, IEEE, 2016.

[57] A. Stanciu and C. Gerigan, "Comparison between implementations efficiency of hls and hdl using operations over galois fields," in *2017 IEEE 23rd International Symposium for Design and Technology in Electronic Packaging (SIITME)*, pp. 171–174, 2017.

[58] G. Akkad, A. Mansour, B. ElHassan, F. L. Roy, and M. Najem, "Fft radix-2 and radix-4 fpga acceleration techniques using hls and hdl for digital communication systems," in *2018 IEEE International Multidisciplinary Conference on Engineering Technology (IMCET)*, pp. 1–5, 2018.

[59] J. Duarte, S. Han, P. Harris, S. Jindariani, E. Kreinar, B. Kreis, J. Ngadiuba, M. Pierini, R. Rivera, N. Tran, *et al.*, "Fast inference of deep neural networks in fpgas for particle physics," *Journal of Instrumentation*, vol. 13, no. 07, p. P07027, 2018.

[60] F. Sánchez, R. Mateos, E. Bueno, J. Mingo, and I. Sanz, "Comparative of hls and hdl implementations of a grid synchronization algorithm," in *IECON 2013 - 39th Annual Conference of the IEEE Industrial Electronics Society*, pp. 2232–2237, 2013.

[61] W. M. Zabołotny, "Implementation of omtf trigger algorithm with high-level synthesis," in *Photonics Applications in Astronomy, Communications, Industry, and High-Energy Physics Experiments 2019*, vol. 11176, p. 1117641, International Society for Optics and Photonics, 2019.

[62] N. Ghanathe, A. Madorsky, H. Lam, D. Acosta, A. George, M. Carver, Y. Xia, A. Jyothishwara, and M. Hansen, "Software and firmware co-development using high-level synthesis," *Journal of Instrumentation*, vol. 12, no. 01, p. C01083, 2017.